

SEEDPRINTS: FINGERPRINTS CAN EVEN TELL WHICH SEED YOUR LARGE LANGUAGE MODEL WAS TRAINED FROM

Anonymous authors
Anonymous submission

ABSTRACT

Fingerprinting Large Language Models (LLMs) underpins provenance verification and model attribution, yet the prevailing methodology rests on a fragile premise: existing fingerprints are evaluated almost exclusively after fine-tuning, once a model has already accreted stable signatures from its training data, optimization dynamics, and hyperparameters. Because the overwhelming majority of a model’s capacity and knowledge is acquired during pretraining rather than downstream fine-tuning, large-scale pretraining—not fine-tuning—constitutes the more fundamental regime in which lineage must be established. We demonstrate that existing fingerprinting methods are *unreliable* precisely in this regime, as the post-hoc signatures they exploit emerge only after substantial training; this dependence violates the classical Galtonian conception of a fingerprint as an identity that is intrinsic and persistent from birth. We therefore introduce a stronger and genuinely intrinsic notion of LLM fingerprinting, **SeedPrints**, which exploits the prediction biases imprinted by random initialization as persistent, seed-dependent identifiers that are present before a single optimization step is taken. We establish that untrained models exhibit reproducible, seed-determined prediction biases, and that these signals—though individually weak—remain statistically detectable throughout training, supporting high-confidence lineage verification. Whereas prior techniques fail during early pretraining or degrade under distribution shift, **SeedPrints** remains effective across the entire training lifecycle, from initialization through large-scale pretraining to downstream adaptation. Experiments on LLaMA-style and Qwen-style models establish seed-level distinguishability and enable birth-to-lifecycle identity verification, while evaluations on large-scale pretraining trajectories and real-world fingerprinting benchmarks confirm robustness under prolonged training, pronounced domain shift, and parameter-altering transformations.

Collectively, these results show that initialization itself imprints a unique and durable identity on an LLM, constituting a true “Galtonian” fingerprint. Code will be released upon acceptance.

1 INTRODUCTION

LLM fingerprints have recently emerged as a principled instrument for identifying, attributing, and tracing large language models through their observable behaviors (Pasquini et al., 2024; Xu et al., 2024; Yoon et al., 2025; Zhang et al., 2024; Zeng et al., 2024). Their purpose is to furnish model owners with a verifiable link between a suspicious model and its putative original, thereby enabling the detection of model theft and unauthorized reuse (Yoon et al., 2025; Zhang, 2025). As foundation models become increasingly valuable assets whose weights are shared, adapted, and redistributed, the reliability of this link—across the full arc of a model’s existence rather than only after deployment—becomes the decisive question.

Much of this literature explicitly borrows the metaphor of biological fingerprints from Francis Galton’s *Finger Prints* (1892) (Galton, 1892), and it is worth taking that metaphor at its word:

SeedPrints vs Baselines: OLMo-2-7B Pre-training Checkpoints

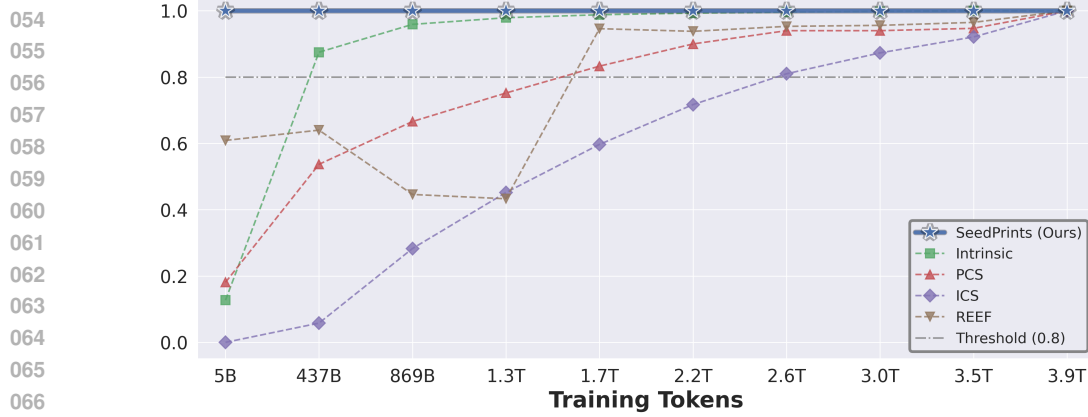


Figure 1: **Existing fingerprinting methods fail to detect model lineage during early pre-training, whereas SeedPrints does not.** We compare five methods on OLMo-2-7B checkpoints spanning 5B to 3.9T training tokens, each tested against the final checkpoint. The y-axis reports the similarity score (higher indicates a stronger lineage signal), and the dashed line marks the 0.8 detection threshold. Every baseline degrades and falls below the threshold at early checkpoints, exposing a systematic blind spot during the pretraining regime; in contrast, **SeedPrints** attains perfect detection ($p \ll 0.001$, plotted as $1 - p$) from the very first checkpoint onward and remains saturated throughout training.

“A fingerprint is the pattern formed by friction-ridge skin on the fingertips; this ridge configuration is individually unique and essentially permanent across an individual’s lifetime.”

The analogy carries a precise and demanding standard: an effective fingerprint must be both unique and persistent, present from the very moment of a model’s “birth” at initialization. Judged against this standard, essentially all existing so-called fingerprinting approaches fall short (Pasquini et al., 2024; Xu et al., 2024; Yoon et al., 2025; Zhang et al., 2024; Zeng et al., 2024; Zhang, 2025; Luan et al., 2025; Tsai et al., 2025; Alhazbi et al., 2025). As Figure 1 makes concrete, these methods fail to reliably detect lineage during early pretraining. The failure is not incidental but structural: because each method is defined only after a model has largely converged—extracting patterns from settled parameters or from generated text—the separability it achieves reflects not a birthmark of the model itself but an imprint of surface-level training components such as data signatures, optimization dynamics, and hyperparameters. This confound has concrete consequences: in Section 5.1, existing baselines are readily misled and fail to identify true lineage when the training data distribution shifts substantially. Such methods therefore operate as *post hoc* identifiers rather than **Galtonian fingerprints**—the innate, immutable marks that accompany a model from its inception.

In this work, we advance a stricter and more principled notion of LLM fingerprinting: an intrinsic property fixed at *initialization* and detectable at *any* point during subsequent training. This reconception directly closes the pretraining blind spot that undermines prior work, and it is substantiated by the following contributions:

- **Exposing a fundamental limitation of existing methods.** We establish that prior fingerprinting approaches cannot reliably detect lineage during early pretraining and are readily misled by substantial shifts in the training data distribution, revealing that their apparent success rests on training-induced rather than intrinsic signals. (Section 5.1)
- **Discovering and operationalizing initialization-driven fingerprints.** We uncover that untrained models already exhibit seed-dependent biases in their internal representations, constituting a weak but persistent identity signal fixed at initialization (Section 3). Building directly on this finding, we introduce **SeedPrints**, a method that isolates these initialization-born fingerprints and, by design, remains robust to confounding factors such as data distribution, training duration, and optimization dynamics. (Section 4)
- **Rigorous empirical validation.** We show that our method separates models differing *only* in initialization seed—even under identical training pipelines and data orders—and that this fingerprint persists under continual training across markedly diverse datasets. (Section 5.1)
- **Strong performance across realistic settings.** Our method reliably detects lineage throughout large-scale pretraining trajectories (e.g., OLMo-2-7B Stage 1) with no early-stage failure, and it performs strongly under large-scale fine-tuning (up to 700B tokens) and practical deployment on

the *LeaFBench* benchmark (Shao et al., 2025), which comprises 65 models spanning 7 mainstream model families and 6 transformations (instruction tuning, fine-tuning, PEFT, quantization, model merging, and distillation). In these settings our method matches the strongest baseline (near-perfect) while substantially outperforming the remainder, thereby uniting birth-stage and deployment-stage verification within a single framework. (Sections 5.2 and 5.3)

2 RELATED WORK

Approaches to LLM protection fall into two complementary families: (i) *watermarking / active fingerprinting* methods that *insert* an identifiable signature into a model or its outputs (Xu et al., 2024; Nasery et al.; Tsai et al., 2025; Sander et al., 2024), and (ii) *passive fingerprinting* methods that *extract* a signature from a model’s pre-existing behaviors without modifying it (Yoon et al., 2025; Zhang et al., 2024; Alhazbi et al., 2025; Pasquini et al., 2024; Suzuki et al., 2025; Zhang, 2025). Our method belongs to the second family but departs from it on a defining axis: the developmental stage at which the identifying signal exists.

Watermarking and Active Fingerprinting. Active approaches deliberately implant verifiable identifiers to enable subsequent ownership checks. Classic techniques include backdoor attacks (Adi et al., 2018; Li et al., 2019; Zhang et al., 2018) and digital signatures or hash functions (Guo & Potkonjak, 2018; Zhu et al., 2020). For language models, two forms predominate: *text watermarks*, which bias generation or insert predefined patterns to encode hidden information (Kirchenbauer et al., 2023; Xu et al., 2024; Nasery et al.); and *model weight watermarks*, which embed identifiers into parameters or bind them to secret triggers through fine-tuning (Luan et al., 2025; Li et al., 2023). Although backdoor-style fingerprints are conceptually simple and can persist through moderate fine-tuning (e.g., Dasgupta et al. 2024), these invasive schemes require control of the training process, which renders them inapplicable to the retroactive marking of third-party models—precisely the setting in which provenance disputes typically arise.

Passive LLM Fingerprinting. Passive fingerprinting, by contrast, identifies models by analyzing their intrinsic properties without any modification, and the available techniques are naturally organized by the level of model access they assume. Under **white-box access**, signatures are extracted directly from model weights, exploiting intrinsic properties such as the distribution of attention matrices (Yoon et al., 2025), the kernel alignment of internal representations (Zhang et al., 2024), or the stable direction of parameter vectors. In the **black-box setting**, fingerprinting instead analyzes input–output behavior, using crafted queries (Pasquini et al., 2024), unique prompt–response pairs (Tsai et al., 2025), stylometry (Alhazbi et al., 2025), or iterative prompting–response games (Iourovitski et al., 2024), though such behavioral signals tend to be less robust to fine-tuning. Crucially, every one of these methods defines its signature *post hoc*: it reads off emergent properties of a *completed* training process rather than the innate, “Galtonian” fingerprint present from random initialization that our work identifies and exploits. Consequently, *they are not designed for lineage verification during large-scale pretraining*, the very regime in which post-training signals have not yet formed—and the gap our method is built to fill.

3 BIASES ORIGINATING FROM INITIALIZATION PERSIST AFTER TRAINING

In this section we establish the empirical foundation of our method: language models exhibit pronounced prediction biases that originate in the random initialization seed, and these biases remain statistically detectable even after training. This observation is the linchpin of the paper, and we present the two components of the argument—that initialization induces seed-specific bias, and that training preserves it—in turn.

Initialization induces seed-specific bias profiles. We evaluate a LLaMA-2–style model initialized with seed 123 on 10,000 random input sequences of length 1,024, with tokens sampled uniformly from the vocabulary. In Figure 2 (Left), we plot the frequency with which each output token (upper, from the final logits) and each hidden dimension (lower, from the last-layer representations) attains the minimum value across the 10,000 random trials. The pronounced departure from uniformity in both plots reveals extreme, systematic output-preference biases rather than incidental fluctuation. Repeating the experiment across initialization seeds isolates the source of this structure: while the overall *magnitude* of bias is stable across seeds, the specific argmin dimensions are seed-determined (Figure 2, Lower Right). These findings are consistent with recent theory showing that biases in randomly initialized models arise from inter-sequence representation contraction, driven by

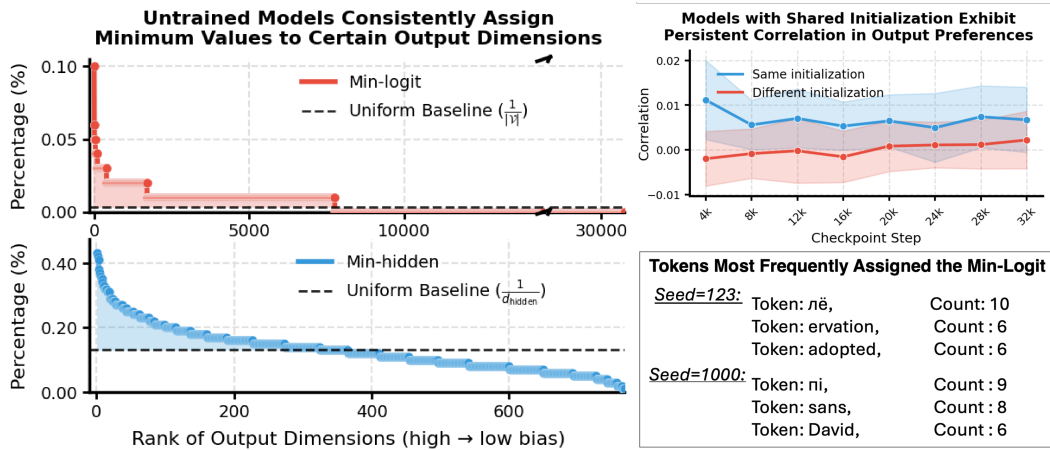


Figure 2: **Initialization-born output bias persists through training.** *Left:* Under completely random inputs, the outputs of a randomly initialized LLaMA-2-style model are far from uniform and instead display clear, systematic bias: certain dimensions are consistently disfavored (i.e., they frequently attain the minimum value across random inputs). This extreme bias is evident in both the logits (top, red) and the final hidden representations (bottom, blue). The dashed line marks the expected frequency under a uniform distribution, and the arrows in the top panel denote a broken x-axis that omits low-frequency tail ranks. *Upper Right:* Throughout training, a model remains weakly but reliably correlated in its output bias with the base model it was trained from. This correlation distribution is consistently shifted upward relative to correlations computed against independently initialized base models, confirming that initialization-born bias leaves a measurable signal that survives training. *Lower Right:* The dimensions exhibiting the strongest bias are seed-determined, illustrated here via the tokens that most frequently receive the minimum logit.

asymmetric nonlinear activations in MLP blocks and further amplified by self-attention (Li et al., 2026). Under this account, distinct initialization seeds induce distinct contraction directions, which in turn produce the distinct argmin patterns we observe across output dimensions—furnishing a mechanistic explanation for why the fingerprint is seed-specific.

Training preserves initialization-born bias. Although training substantially rescales output magnitudes, the relative preference over output dimensions imprinted at initialization is not erased. We train the previously initialized model (seed 123) for one epoch on OpenWebText (Gokaslan et al., 2019) and evaluate intermediate checkpoints. We focus on the $m = 50$ output dimensions most frequently assigned the minimum value by the initialized model, and for each checkpoint we compute the correlation between its responses and those of the initialized model across random inputs, restricted to these dimensions; intuitively, this quantifies how similarly the checkpoint and its initialization rank—and disfavor—these coordinates across inputs. As a control, we compute the identical correlation against an independently initialized model (seed 1000). As shown in Figure 2 (Upper Right), although the absolute correlations are small, correlations for models sharing an initialization are consistently and systematically shifted upward relative to the independent control, and they stabilize rather than decaying toward zero as training proceeds. Taken together, these results demonstrate that initialization leaves a weak but statistically detectable bias signal that persists throughout training—the property that makes it usable as a lifelong fingerprint.

4 ALGORITHM

Section 3 establishes that models sharing an initialization lineage retain weak but consistent agreement on a subset of output dimensions—specifically, those most disfavored by the untrained model under random inputs. This motivates a concrete design objective: given any two models, and *without* access to the true initialized model, recover the set of dimensions that encode this bias signal and test whether the models’ agreement on them is statistically significant. The complete procedure is summarized in Algorithm 1; we specify each component in turn below.

Extracting identity dimensions between any two models. Let $X = \{x_i\}_{i=1}^n$, where each $x_i \in \mathbb{R}^{\ell \times d}$ denotes a random sequence of length ℓ . Each random input x_i may be instantiated in one of two equivalent ways: by uniformly sampling ℓ token IDs from the vocabulary, or by directly sampling ℓ independent vectors from a d -dimensional isotropic Gaussian distribution. For any model g , we define

its mean response vector as $\bar{g} := \frac{1}{n} \sum_{i=1}^n g(x_i) \in \mathbb{R}^{d_{\text{out}}}$, where d_{out} denotes the output dimensionality and $g(x_i)$ denotes either the model’s logits or its final-layer representations. We then define its disfavored dimensions as the set of bottom- m coordinates of $\bar{g}$:

$$\mathcal{M}_g := \arg \min_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{j \in J} \bar{g}_j.$$

We adopt the mean response vector as the bias estimator because it is provably more stable and noise-robust than per-sample argmin frequency (i.e., counting how often each dimension attains the minimum value, as illustrated in Figure 2), while also performing at least as well empirically; a formal comparison is given in Appendix C.1.

Given two models f and f' , we define their *identity dimensions* as the intersection $S := \mathcal{M}_f \cap \mathcal{M}_{f'}$. The rationale is direct: two models that share an initialization lineage should independently recover a similar set of disfavored dimensions, yielding non-trivial overlap in their bottom- m sets. The intersection criterion thus functions as a form of mutual verification—it suppresses spurious correlations arising from architectural similarity or shared training dynamics and isolates the bias signal most plausibly attributable to shared initialization. When one model lies closer to initialization, its disfavored dimensions may equivalently serve as a direct anchor.

Correlation statistics. Restricting both models to the identity dimensions S , we compare their responses across random inputs.¹ For each $j \in S$, we compute a Kendall–Tau rank correlation:

$$\tau_j = \text{KendallTau}(\{f(x_i)_j\}_{i=1}^n, \{f'(x_i)_j\}_{i=1}^n).$$

This yields a set of correlation statistics $T = \{\tau_j\}_{j \in S}$ characterizing the distribution of agreement between the two models across identity dimensions. Consistent with Section 3, same-lineage models exhibit distributionally higher agreement on these dimensions than unrelated models.

Hypothesis testing via a null distribution. Because the correlation distributions of same-lineage and different-lineage models can overlap, thresholding an individual statistic is inadequate; we instead test whether the observed correlations are significantly larger than expected under the null hypothesis of no lineage relation. Ideally this null would be estimated from the correlation distribution between independent models, as in Figure 2. Estimating such a baseline empirically, however, demands comparisons across many independently initialized models and is therefore computationally expensive. Appendix B shows that the null correlation distribution between independently initialized models is well approximated by a Gaussian, which motivates a substantially cheaper alternative: an empirical null constructed by passing Gaussian surrogates through the identical pipeline, as summarized in Algorithm 1.

This simulation-based null already yields a strong empirical approximation, but it retains avoidable randomness and computational overhead—its precision is bounded by the number of simulation trials, its estimates fluctuate across runs, and comparing two empirical distributions typically imposes additional assumptions from the chosen test (e.g., the one-sided t -test or the Mann–Whitney U test). Since the null distribution of Kendall–Tau under weak independence is available in closed form, we go one step further and replace the simulation with an analytical test, while shipping both empirical and analytical implementations in our code repository. Concretely, under the null each Kendall–Tau correlation is approximately zero-mean with known variance σ^2 . By the central limit theorem, as the number of identity dimensions $|S|$ grows, the distribution of $\bar{\tau} = \frac{1}{|S|} \sum_{j \in S} \tau_j$ is well approximated by $\bar{\tau} \sim \mathcal{N}\left(0, \frac{\sigma^2}{|S|}\right)$. Although softmax normalization introduces weak cross-dimension dependencies, the use of a relatively high temperature ($T = 10$) attenuates these effects, rendering the independence assumption an accurate approximation in practice; we empirically confirm that this Gaussian approximation closely matches the distribution obtained from the full pipeline (Appendix B). We accordingly compute the one-sided z -score $z = \frac{\bar{\tau}}{\sigma/\sqrt{|S|}}$ and the corresponding p -value $p = 1 - \Phi(z)$, where Φ is the standard normal cumulative distribution function, and we reject the null—declaring shared lineage—whenever $p < 0.01$. The result is a calibrated statistical decision rather than a heuristic similarity comparison.

¹Before computing the correlations, we apply a row-wise softmax normalization to each sample’s output vector. This removes scale differences across models and dimensions while preserving the relative ordering that carries identity information for rank-based statistics such as Kendall–Tau. A relatively high temperature is used to avoid degenerate one-hot behavior.

Algorithm 1 Distribution Correlation Test on Identity Dimensions

Require: base model f , suspicious model f' ; random inputs $X = \{x_i\}_{i=1}^n$; fingerprint size m ; significance level α

► **Step 1: Localize biased dimensions**

- 1: Compute average outputs $\bar{f}, \bar{f}'$ over X
- 2: $\mathcal{M}(f), \mathcal{M}(f') \leftarrow$ bottom- m dimensions of $\bar{f}, \bar{f}'$
- 3: $\mathcal{S} \leftarrow \mathcal{M}(f) \cap \mathcal{M}(f')$ (identity dimensions)

► **Step 2: Form correlation distribution**

- 4: Normalize $f(x_i)_{\mathcal{S}}, f'(x_i)_{\mathcal{S}}$ for each $x_i \in X$ across identity dimension
- 5: **for** each $s_j \in \mathcal{S}$ **do**
- 6: $\tau_j \leftarrow \text{KendallTau}(\{f(x_i)_{s_j}\}_{i=1}^n, \{f'(x_i)_{s_j}\}_{i=1}^n)$ (correlation in in input-wise preferences)
- 7: **end for**
- 8: $\mathcal{T} \leftarrow \{\tau_j\}_{j=1}^{|\mathcal{S}|}$

► **Step 3: Hypothesis test against null**

- 9: Construct $\mathcal{T}_{\text{null}}$ by applying the same pipeline to two Gaussian matrices $Y^{(1)}, Y^{(2)} \sim \mathcal{N}(0, I)^{N \times d_{\text{out}}}$
- 10: Test $H_0 : \mathcal{T} = \mathcal{T}_{\text{null}}$ vs. $H_1 : \mathcal{T} > \mathcal{T}_{\text{null}}$
- 11: **Return** SameLineage $\leftarrow \mathbf{1}(p\text{-value} < \alpha)$

Practical instantiation. Model outputs may be taken as either logits or hidden representations, and both are consistent with our formulation. However, logits require substantially more random inputs to reliably estimate bias patterns as the vocabulary grows, incurring considerable computational overhead (Appendix C.2 provides the supporting theoretical analysis). For efficiency and stability, we therefore adopt final-layer hidden states as the default model output. The resulting p -value constitutes a direct statistical test for lineage, obviating any reliance on heuristic similarity thresholds.

5 EXPERIMENTS

Our experiments are organized around two complementary claims. In Section 5.1 we establish that our method functions as a *genuine* fingerprint by demonstrating both properties the Galtonian standard requires: (i) *birth verification at the seed level*, and (ii) *verifiability across the entire training lifecycle*. Existing baselines satisfy neither—they cannot support verification at early pretraining stages and rely predominantly on data-dependent signals that collapse under substantial distribution shift. Unless stated otherwise, we report results using the analytical null with *hidden state* outputs; additional results using the empirical null with both *logit* and *hidden state* outputs, evaluated under the one-sided t -test and the Mann–Whitney U test, are provided in Appendix D.2 and corroborate the same conclusions. In Section 5.3 we then evaluate all methods under practical infringement scenarios using *LeafBench* (Shao et al., 2025): across 65 distinct model instances spanning 6 representative post-development techniques, our method matches the strongest baselines in post-training settings while remaining robust to the full range of deployment transformations.

Baselines. We principally compare against four passive fingerprinting baselines (weight- or representation-based). *Intrinsic fingerprint* (Yoon et al., 2025) (denoted *PDF* in some papers) compares models via the similarity of the layerwise standard-deviation profiles of their attention parameters. *REEF* (Zhang et al., 2024) computes centered-kernel-alignment (CKA) similarity between the feature representations elicited by identical samples across two models. *PCS* and *ICS* (Zeng et al., 2024) (collectively *HuRef* in some papers) are weight-similarity methods: PCS flattens all parameters and measures cosine similarity, whereas ICS constructs invariant terms from the weights and measures cosine similarity on those invariants. In Section 5.3 we additionally include a (weaker) gradient-based fingerprint, *Gradient* (Shao et al., 2025). Following Zhang et al. (2024), all similarity-based baselines use a **0.8 similarity threshold** for binary decisions.

For readability, cell colors in every experiment table encode ground-truth lineage: **green** marks models that share a source, and **red** marks models from distinct sources. For instance, s_{42}^{init} vs. s_{42}^{base} compares a seed-42 initialization against its continued-pretrained descendant, and is thus green; by contrast, s_{2000}^{init} vs. s_{42}^{base} compares a seed-2000 initialization against a model trained from a seed-42 initialization, and is thus red. A ✓ denotes a correct detection and a ✗ denotes an error.

5.1 BIRTH-TO-LIFECYCLE “BIOMETRIC” FINGERPRINTING

We train 12-layer, 12-head LLaMA-style models (Touvron et al., 2023) with RoPE (Su et al., 2021) and Qwen-style models (Team, 2024) from scratch. The main paper reports LLaMA-style results and

Table 1: Fingerprint behaviors between models initialized with different seeds: all p -values exceed 0.01, so SeedPrints never conflates distinct-seed models.

Model Pairs	p -value (> 0.01)
s_{42} VS. s_{2000}	0.116
s_{123} VS. s_{42}	1.000
s_{1000} VS. s_{123}	0.327
s_{2000} VS. s_{1000}	1.000

Table 3: Identical dataset and training order do not force fingerprint behaviors to coincide across different initializations: all p -values remain > 0.01 .

Model Pair	p -value (> 0.01)
s_{42}^{init} VS. s_{123}^{base}	0.573
s_{123}^{init} VS. s_{2000}^{base}	0.724
s_{2000}^{init} VS. s_{1000}^{base}	0.109
s_{1000}^{init} VS. s_{42}^{base}	0.883

Table 2: Trained models retain the fingerprint of their initialization: SeedPrints yields $p < 0.01$ for every s^{init} - s^{base} pair, whereas all four baselines fail to detect the shared lineage.

Model Pairs	SeedPrints p -value	Baselines			
		Intrinsic	REEF	PCS	ICS
s_{42}^{init} VS. s_{42}^{base}	3.37e-26✓	-0.021×	0.375×	0.580×	0.196×
s_{123}^{init} VS. s_{123}^{base}	3.16e-33✓	0.149×	0.369×	0.581×	0.188×
s_{1000}^{init} VS. s_{1000}^{base}	1.12e-21✓	-0.252×	0.381×	0.581×	0.188×
s_{2000}^{init} VS. s_{2000}^{base}	3.70e-31✓	-0.337×	0.331×	0.581×	0.188×

Table 4: Fingerprint persistence under continual training on diverse datasets (base model: seed 1000, corpus openwebtext). SeedPrints attributes lineage correctly under both TinyStories and the sharply shifted code corpus (The Stack), whereas the baselines break down on The Stack.

Setting	SeedPrints p -value	Baselines			
		Intrinsic	REEF	PCS	ICS
Continual corpus (seed)					
TinyStories (1000)	≈ 0 ✓	1.000✓	0.759×	0.999✓	0.996✓
TinyStories (123)	1.000✓	0.950×	0.658✓	0.332✓	0.012✓
the_stack (1000)	≈ 0 ✓	0.489×	0.557×	0.585×	0.123×
the_stack (123)	1.000✓	0.445✓	0.580✓	0.301✓	0.026✓

defers Qwen-style results to Appendix D; the conclusions are consistent across both architectures, indicating that our findings are not artifacts of a particular model design.

Different initialization seeds produce distinct fingerprints. Table 1 reports p -values from our correlation test between pairs of models initialized with different random seeds (42, 123, 1000, and 2000). Every p -value is > 0.01 , so our method never conflates models originating from different seeds: distinct seeds yield distinct fingerprint behaviors, and models can be separated “at birth.”

Training preserves the initialization fingerprint. Table 2 compares each initialization model s^{init} with its descendant s^{base} trained on OpenWebText (Gokaslan et al., 2019) (≈ 10 B tokens). Across all seed-model pairs the p -values are uniformly < 0.01 , demonstrating that the bias profiles remain strongly correlated and that descendant and initialization share a common lineage—in short, the trained model inherits the fingerprint of its initialization. The baselines exhibit the opposite behavior: without exception they fail to distinguish across seeds, which confirms that their separability derives from training-induced artifacts rather than from initialization itself.

Identical data and order do not force fingerprints to converge. In Table 3, all four “suspicious” models s_i^{base} for $i \in \{42, 123, 100, 2000\}$ are trained on *exactly the same corpus (OpenWebText) and in the same data order*—we fix the training seed to lock the curriculum—so that their sole point of difference is the initialization seed i . Across all cross-seed pairs the p -values remain > 0.01 , in sharp contrast to the near-zero values of Table 2. The fingerprint is therefore attributable to the seed alone: it remains seed-specific even when data and curriculum are held identical.

Continual training on diverse datasets does not confound the fingerprint. The experiments above serve to establish that SeedPrints behaves as a biometric fingerprint. From a copyright standpoint, the inability of prior methods to separate models by seed is not itself a shortcoming, since initialization seeds carry no clear copyright status. Their genuine fragility lies elsewhere: their attribution is easily misled by *data distribution, failing to recognize lineage when the training distribution shifts substantially during continued training*.

To isolate this failure mode, in Table 4 we continue training a base model (seed 1000, pretrained on OpenWebText (Gokaslan et al., 2019)) on two deliberately dissimilar corpora: TinyStories (Eldan & Li, 2023) (synthetic children’s stories) and The Stack (Kocetkov et al., 2022) (permissively licensed GitHub code). We contrast (i) true descendants trained from the base against (ii) *distractors* derived from a different base model (seed 123). Because the baselines are insensitive to initialization differences, the distractor base is pretrained with a different data order on OpenWebText and then

Table 5: Fingerprinting results under large-scale finetuning. Each row compares a target model against LLaMA-2-7B. **SeedPrints** reports the p -value from our correlation test (< 0.01 indicates a strong lineage signal), while the four baselines report similarity scores (threshold = 0.8, higher is better). SeedPrints maintains decisive detection ($p < 0.01$) across the full 5M-to-700B-token range, including the 500B- and 700B-token regimes where PCS and ICS fail.

Model	# Tokens	SeedPrints p	Intrinsic $\uparrow$	REEF ($\uparrow$)	PCS ($\uparrow$)	ICS ($\uparrow$)
Llama-2-finance-7B (Heenan, 2023)	5M	10^{-41955} ✓	1.0000 ✓	0.9950 ✓	0.9979 ✓	0.9952 ✓
Vicuna-1.5-7B (Chiang et al., 2023)	370M	$10^{-103043}$ ✓	1.0000 ✓	0.9985 ✓	0.9985 ✓	0.9949 ✓
Wizardmath-7B (Luo et al., 2023)	1.8B	$10^{-180550}$ ✓	1.0000 ✓	0.9979 ✓	1.0000 ✓	0.9994 ✓
Meditron-7B (Chen et al., 2023)	48B	10^{-42566} ✓	0.9990 ✓	0.9978 ✓	1.0000 ✓	0.9817 ✓
CodeLlama-7B (Meta AI, 2024)	500B	10^{-3552} ✓	0.9480 ✓	0.9947 ✓	0.6863 ✗	0.3369 ✗
Llemma-7B (Azerbayev et al., 2023)	700B	10^{-5136} ✓	0.9470 ✗	0.9984 ✓	0.6682 ✗	0.2905 ✗

continually trained on the *same* target corpus, so that the decisive question is whether an attribution method can identify which descendant genuinely shares lineage with the base.

The result is unambiguous: every prior baseline fails in the code setting (The Stack), misclassifying true descendants as distractors. This exposes that the baselines largely track *domain* similarity rather than lineage identity—TinyStories remains close in distribution to the pretraining corpus (OpenWebText), whereas The Stack diverges sharply, and such a pronounced shift suffices to defeat detection. Our method, by contrast, attributes lineage correctly across both corpora. SeedPrints is thus not a proxy for data distribution: it withstands substantial domain shift and persists well beyond the initial pretraining stage.

5.2 ALL-STAGE VERIFIABLE FINGERPRINTS

Controlled pretraining trajectory. Can our method reliably recognize its own offspring along an entire training trajectory? We first answer this in a controlled pretraining experiment on OpenWebText, in which the initialization seed is known: testing lineage at intermediate checkpoints yields consistent detection at every stage (Appendix D.2.1, Figure 5).

Large-scale pretraining. To assess lineage under realistic foundation-model training, we analyze ten checkpoints from the OLMo-2-7B (OLMo et al., 2024) Stage-1 run (5B $\rightarrow$ 3.9T tokens), treating the final checkpoint as the target model and every earlier checkpoint as a tested ancestor. Unlike the controlled OpenWebText setting, this run spans heterogeneous corpora, far longer optimization, and large-scale representation drift—a stringent stress test of persistence. We plot $1 - p$ for directional consistency with the similarity-based baselines; all of our p -values are $\ll 0.01$ and numerically indistinguishable from zero.

Figure 1 delivers a clear verdict: **every existing baseline fails to verify lineage during early pretraining** (e.g., $ICS \approx 0$, $PCS < 0.3$, and $REEF < 0.6$ within the first **trillion** tokens), whereas SeedPrints remains verifiable from the outset and only strengthens as training proceeds. This exposes a critical and previously overlooked gap in existing evaluations: **prior work verifies lineage almost exclusively after fine-tuning, where detection is substantially easier**. Our findings show that large-scale pretraining itself can dramatically reshape representations and *inflate* late-stage lineage signals, conferring a *false sense of safety*. We therefore contend that lineage verification must prioritize the early stages of pretraining—when misuse is hardest to detect—rather than relying on late-stage checkpoints alone.

Large-scale finetuning. We further benchmark our method against the baselines under standard fine-tuning evaluations, testing suspect models fine-tuned from Llama-2-7b (the base model) at data volumes ranging from 5 million to 700 billion tokens. The suspects span a deliberately diverse set of downstream variants—Llama-2-finance-7b (Heenan, 2023), Vicuna-1.5-7b (Chiang et al., 2023), WizardMath-7b (Luo et al., 2023), Chinese-LLaMA-2-7b (Chen et al., 2023), Code-Llama-7b (Meta AI, 2024), and Llemma-7b (Azerbayev et al., 2023)—with fine-tuning volumes of 5M, 370M, 1.8B, 13B, 500B, and 700B tokens, respectively. As Table 5 shows, our method maintains $p < 0.01$ in every setting, sustaining decisive lineage detection even under fine-tuning budgets as large as 700B tokens, where two of the weight-based baselines break down.

5.3 ROBUSTNESS UNDER REALISTIC DEPLOYMENTS

Real-world deployments routinely subject foundation models to *parameter-altering* adaptations—fine-tuning, PEFT, quantization, and beyond—that can weaken or distort fingerprint signals, and our method is designed to remain effective precisely in this regime. To evaluate its reliability under these realistic conditions, we adopt *LeaFBench* (Shao et al., 2025), a benchmark for language-model copyright auditing.

Across Source Models. We consider two categories of source model for auditing: *Pre-Trained (PT)* and *Instruction-Tuned (IT)*. For each category, the task is to separate *derivative* models (produced by post-training adaptation) from *independent* models (trained without the source weights), mirroring the common auditing scenario in which a lineage claim must be tested against either a PT base or its IT variant. The evaluation spans 65 models in total; the complete list of model names and HuggingFace repositories appears in Appendix E.

Metrics. We quantify detection performance with (i) the Area Under the ROC Curve (AUC) and (ii) the Kolmogorov–Smirnov (KS) statistic (Berger & Zhou, 2014), the latter measuring the maximum separation between the score distributions of derivative and independent models.

Our method emits per-model p -values from statistical tests, which are fundamentally different objects from the similarity scores used by the baselines: similarity scores are linear quantities whose differences carry proportional meaning, whereas p -values are tail probabilities encoding the rarity of an observation under the null and cannot be read on a linear scale. To enable comparison with baselines that compute AUC from similarity scores, we convert our p -values via $s = 1 - p$. This conversion is, if anything, unfavorable to our method—small p -values vary on an exponential scale while threshold sweeping is linear, which blurs fine-grained differences—yet even under this handicap our approach attains comparable or superior performance. For evaluation we therefore (i) compute AUC for all methods by sweeping thresholds over the scores s and (ii) report the KS statistic as a threshold-free measure of distributional separability. This protocol preserves compatibility with the baseline pipelines while retaining the statistical interpretability of our method. We omit Manhattan distance, as absolute distances between p -values are not meaningful.

It bears emphasis that calibrating an optimal similarity threshold is infeasible in most practical settings. We report AUC purely for comparability, but the essential distinction remains: our method delivers a direct statistical test. **For any pair of models, it outputs a p -value that supports a definitive, threshold-free decision on whether they share lineage**, which is what enables reliable verification in deployment.

Parameter-Altering Techniques. To probe fingerprinting robustness across the transformation landscape, we evaluate models produced via (1) instruction tuning (Instruct), (2) general-purpose fine-tuning (Finetune), (3) parameter-efficient fine-tuning (PEFT), (4) quantization, (5) model merging (Merge), and (6) distillation.

5.3.1 FAMILY-WISE EFFECTIVENESS

The benchmark comprises seven model families. For each family we treat both its pretrained base model (PT) and its instruction-tuned model (IT) as separate source models and compare them against all other models in the pool of 65, with results reported in Table 6. We omit the TinyLlama-1.1B families from Table 6, as they are trivial for every method. Each column corresponds to a single family (AUC computed over the corresponding tests with a family-calibrated threshold), whereas the overall score is computed across all test pairs under a single global threshold; the overall score is therefore *not* the average of the per-family scores. Three methods—*Intrinsic*, *ICS*, and our *SeedPrints*—are effectively saturated across all alterations (overall AUC of 0.994, 0.994, and 0.992; KS of 0.989, 0.943, and 0.986, respectively), attaining the maximal per-family score in most cases, while the remaining baselines, *REEF* and *Gradient*, trail substantially. This is a notable result: although *SeedPrints* is designed to capture *intrinsic seed-level* fingerprints, it simultaneously operates as a biometric-grade identifier that delivers reliable, persistent identity tracking across diverse model families—matching the best deployment-oriented baselines on their own terms.

5.3.2 ALTERATION-WISE ROBUSTNESS

Across alteration types, *Instruct* is consistently the easiest case (even the weakest baseline, *Gradient*, reaches AUC 0.895), whereas *Finetune* and *Merge* are the most demanding overall—for *Finetune*

Table 6: Performance comparison of LLM fingerprinting methods across source models. “PT” and “IT” denote pre-trained and instruction-tuned source models, respectively. The per-family breakdown covers the 6 families possessing both PT and IT models. The “Overall” scores are computed over all test model pairs under a single global threshold, and are therefore not the average of the per-family scores. SeedPrints reaches near-saturated overall AUC/KS (0.992/0.986), on par with the strongest baselines (Intrinsic, ICS) and far above REEF and Gradient.

Method	Metric	Qwen2.5-7B		Qwen2.5-14B		Llama3.1-8B		Mistral-7B		Gemma2-2B		Llama2-7B		Overall
		PT	IT	PT	IT	PT	IT	PT	IT	PT	IT	PT	IT	
REEF	AUC $\uparrow$	0.796	0.862	0.947	0.913	1.000	0.999	0.997	0.997	0.963	0.968	0.750	0.706	0.915
	KS Statistic $\uparrow$	0.492	0.633	0.875	0.735	0.987	0.987	0.975	0.980	0.950	0.837	0.750	0.694	0.739
Gradient	AUC $\uparrow$	0.657	0.743	0.763	0.789	0.897	0.904	0.872	0.682	1.000	1.000	0.650	0.768	0.801
	KS Statistic $\uparrow$	0.354	0.423	0.500	0.563	0.646	0.705	0.725	0.500	1.000	1.000	0.275	0.479	0.508
ICS	AUC $\uparrow$	1.000	1.000	0.997	0.997	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.994
	KS Statistic $\uparrow$	1.000	1.000	0.975	0.979	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.943
Intrinsic	AUC $\uparrow$	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.994
	KS Statistic $\uparrow$	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.989
SeedPrints	AUC $\uparrow$	0.992	0.994	0.988	0.990	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.992
	KS Statistic $\uparrow$	0.985	0.987	0.975	0.980	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.986

Table 7: Performance of LLM fingerprinting methods across parameter-altering techniques. SeedPrints stays robust across all six transformations (AUC $\geq$ 0.959), with its only appreciable dip on *Merge*, and attains an overall AUC/KS of 0.991/0.985—closely tracking the strongest baselines while additionally supporting birth-stage verification.

Method	Metric	Instruct	Finetune	PEFT	Quantization	Merge	Distillation	Overall Avg
REEF	AUC $\uparrow$	0.996	0.872	0.994	0.871	0.954	0.858	0.924
	KS Statistic $\uparrow$	0.949	0.637	0.940	0.812	0.851	0.615	0.801
Gradient	AUC $\uparrow$	0.895	0.729	0.776	0.756	0.832	0.887	0.812
	KS Statistic $\uparrow$	0.756	0.441	0.469	0.586	0.519	0.679	0.575
ICS	AUC $\uparrow$	1.000	0.987	1.000	1.000	0.981	1.000	0.995
	KS Statistic $\uparrow$	1.000	0.925	1.000	1.000	0.941	1.000	0.978
Intrinsic	AUC $\uparrow$	1.000	0.980	1.000	1.000	1.000	1.000	0.997
	KS Statistic $\uparrow$	1.000	0.971	1.000	1.000	1.000	1.000	0.995
SeedPrints	AUC $\uparrow$	1.000	0.995	0.990	1.000	0.959	1.000	0.991
	KS Statistic $\uparrow$	1.000	0.990	0.980	1.000	0.941	1.000	0.985

no method attains perfect performance. The remaining transformations yield a more mixed picture: REEF drops markedly under *Quantization* (0.996 $\rightarrow$ 0.871) and *Distillation* (0.996 $\rightarrow$ 0.858) while remaining resilient to *PEFT* (0.994), whereas *PEFT* and *Quantization* are the harder cases for Gradient (0.895 $\rightarrow$ 0.776/0.756). The three stronger methods are broadly robust across all transformations (all scores $>$ 0.9). SeedPrints declines only slightly to 0.959 on *Merge*, corresponding to a single misclassified merge model out of seventeen; this lone error indicates that weight-space interpolation with unrelated model weights can, in isolated cases, dilute the initialization signal toward the detection threshold. Averaged across all six alteration types, Intrinsic records the highest overall AUC, followed closely by ICS and SeedPrints—leaving our method within a negligible margin of the best deployment-specialized baselines while additionally providing the birth-stage guarantees that they lack.

5.4 ABLATION STUDY ON HYPERPARAMETERS

Our method exposes three hyperparameters: the number of random input sequences n , the length of each random sequence l , and the fingerprint size m . The first two jointly govern the total volume of random input, while m sets how many low-mean coordinates enter the test. Throughout the main experiments in Sections 5.2 and 5.3 we use $n=2000$, $l=1024$, and $m=0.1d_{\text{out}}$, where d_{out} is the hidden-state dimensionality—a specification that adapts automatically across open-source models of differing width. Here we vary each hyperparameter in turn while holding the others at their defaults ($n = 2000$, $m = 400$, $l = 768$), reporting empirical accuracy and false-positive rate at a fixed significance level $\alpha = 0.01$ in Table 8.

Increasing either n or l consistently improves identification performance, a trend that our theoretical analysis in Appendix C.2 predicts exactly: accurate recovery of the population-level bias requires the number of queries to satisfy $n = \Omega\left(\frac{\log d_{\text{out}}}{\gamma^2}\right)$, where γ is the bias margin separating the biased coordinates from the remainder. A given output dimension thus demands a minimum sample count for reliable detection, which accounts for the degraded performance at small n . The margin γ in turn grows with l through accumulation effects, so longer sequences sharpen the separation; consequently, when the number of sequences is fixed and insufficient, increasing l lowers the effective query requirement and restores detection.

The effect of m is, by contrast, non-monotonic. When m is too small, the selected coordinates are unstable because there are too few empirical observations to form a reliable distribution; when m is too large, the many irrelevant coordinates admitted into the set inject noise that shrinks γ , which would demand quadratically more samples to offset. Empirically, m in the range 200–400 performs well across model sizes, consistent with our heuristic $m \approx 0.1d_{\text{out}}$ for the typical range $d_{\text{out}} \in [3000, 5000]$.

6 CONCLUSION

We introduced *SeedPrints*, a stronger and genuinely intrinsic notion of LLM fingerprinting that traces a model’s lineage all the way back to its random initialization. Our central finding is that initialization itself leaves a persistent “Galtonian” fingerprint on neural language models: untrained models already exhibit stable, seed-dependent token-preference patterns, and these biases persist—statistically and measurably—throughout training. Building directly on this finding, we developed a distribution-correlation test over *identity dimensions* that certifies shared lineage with calibrated statistical significance rather than a heuristic threshold. Across LLaMA- and Qwen-style families and a broad suite of realistic deployments—continued pretraining on divergent corpora, instruction tuning, PEFT, quantization, merging, and distillation—SeedPrints delivers “birth-to-lifecycle” verification and withstands substantial domain shift, providing a practical and reliable tool for provenance and copyright auditing.

Limitations and Future Directions Our objective in this work is to uncover fingerprint signals rooted in initialization-stage bias, and the method accordingly requires access to model outputs (logits or hidden states), so it may not apply when only restricted API access is available. We view this boundary in context: the pursuit of *true fingerprints* for LLMs remains at an early stage, and most existing approaches—especially black-box ones—depend on signals that surface only in later training or from alignment-driven artifacts, and therefore cannot robustly protect models at the pretrained stage. This makes reliable all-stage fingerprints all the more urgently needed, and it is exactly the gap our method begins to close. Full discussions appear in Appendix A.2.

Table 8: Hyperparameter ablation at significance level $\alpha = 0.01$: accuracy improves with larger n and l , while m exhibits the predicted non-monotonic behavior.

Ablation on n	200	500	2000
ACC	0.7368	0.7778	0.9375
Empirical FPR	0	0	0
Ablation on l	512	768	1024
ACC	0.9048	0.9375	1.0000
Empirical FPR	0.04	0	0
Ablation on m	200	400	800
ACC	0.9375	0.9375	0.875
Empirical FPR	0.04	0	0.1463

REFERENCES

- Yossi Adi, Carsten Baum, Moustapha Cisse, Benny Pinkas, and Joseph Keshet. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th USENIX security symposium (USENIX Security 18)*, pp. 1615–1631, 2018.
- Saeif Alhazbi, Ahmed Hussain, Gabriele Oligeri, and Panos Papadimitratos. Llms have rhythm: Fingerprinting large language models using inter-token times and network traffic analysis. *IEEE Open Journal of the Communications Society*, 2025.
- Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. *arXiv preprint arXiv:2310.10631*, 2023.
- Vance W Berger and YanYan Zhou. Kolmogorov–smirnov test: Overview. *Wiley statsref: Statistics reference online*, 2014.
- Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramer. Membership inference attacks from first principles. In *2022 IEEE symposium on security and privacy (SP)*, pp. 1897–1914. IEEE, 2022.
- Zeming Chen, Alejandro Hernández-Cano, Angelika Romanou, Antoine Bonnet, Kyle Matoba, Francesco Salvi, Matteo Pagliardini, Simin Fan, Andreas Köpf, Amirkeivan Mohtashami, Alexandre Sallinen, Alireza Sakhaeirad, Vinitra Swamy, Igor Krawczuk, Deniz Bayazit, Axel Marmet, Syrielle Montariol, Mary-Anne Hartley, Martin Jaggi, and Antoine Bosselut. Meditron-70b: Scaling medical pretraining for large language models, 2023.
- Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Agnibh Dasgupta, Abdullah Tanvir, and Xin Zhong. Watermarking language models through language models. *arXiv preprint arXiv:2411.05091*, 2024.
- Ronen Eldan and Yuanzhi Li. Tinstories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.
- Francis Galton. *Finger prints*. Number 57490-57492. Cosimo Classics, 1892.
- Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. Openwebtext corpus. <http://Skyllion007.github.io/OpenWebTextCorpus>, 2019.
- Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>, 2022.
- Jia Guo and Miodrag Potkonjak. Watermarking deep neural networks for embedded systems. In *2018 IEEE/ACM international conference on computer-aided design (ICCAD)*, pp. 1–8. IEEE, 2018.
- Collin Heenan. Llama2-7b-finance (hugging face model). <https://huggingface.co/cxllin/Llama2-7b-Finance>, 2023. Fine-tuned from NousResearch/Llama-2-7b-hf; MIT License; accessed 2025-09-02.
- Dmitri Iourovitski, Sanat Sharma, and Rakshak Talwar. Hide and seek: Fingerprinting large language models with evolutionary learning. *arXiv preprint arXiv:2408.02871*, 2024.
- John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *International Conference on Machine Learning*, pp. 17061–17084. PMLR, 2023.
- Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 tb of permissively licensed source code. *Transactions on Machine Learning Research (TMLR), Preprint*, 2022.

- Jaehoon Lee, Lechao Xiao, Samuel Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent. *Advances in neural information processing systems*, 32, 2019.
- Peixuan Li, Pengzhou Cheng, Fangqi Li, Wei Du, Haodong Zhao, and Gongshen Liu. Plmmark: a secure and robust black-box watermarking framework for pre-trained language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 14991–14999, 2023.
- Siquan Li, Yao Tong, Haonan Wang, and Tianyang Hu. Transformers are born biased: Structural inductive biases at random initialization and their practical consequences. *arXiv preprint arXiv:2602.05927*, 2026.
- Zheng Li, Chengyu Hu, Yang Zhang, and Shanqing Guo. How to prove your model belongs to you: A blind-watermark based framework to protect intellectual property of dnn. In *Proceedings of the 35th annual computer security applications conference*, pp. 126–137, 2019.
- Xiaokun Luan, Zeming Wei, Yihao Zhang, and Meng Sun. Robust and efficient watermarking of large language models using error correction codes. *Proceedings on Privacy Enhancing Technologies*, 2025.
- Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*, 2023.
- Meta AI. Codellama-7b-hf: Code llama base 7b model (hugging face). <https://huggingface.co/codellama/CodeLlama-7b-hf>, 2024. Base 7 billion-parameter Code Llama model for code synthesis and understanding; trained between January and July 2023; licensed under Meta Llama 2 license; accessed 2025-09-02.
- Anshul Nasery, Jonathan Hayase, Creston Brooks, Peiyao Sheng, Himanshu Tyagi, Pramod Viswanath, and Sewoong Oh. Scalable fingerprinting of large language models. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*.
- Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 olmo 2 furious, 2024. URL <https://arxiv.org/abs/2501.00656>.
- Dario Pasquini, Evgenios M Kornaropoulos, and Giuseppe Ateniese. Llmmap: Fingerprinting for large language models. *arXiv preprint arXiv:2407.15847*, 2024.
- Tom Sander, Pierre Fernandez, Alain Durmus, Matthijs Douze, and Teddy Furon. Watermarking makes language models radioactive. *Advances in Neural Information Processing Systems*, 37: 21079–21113, 2024.
- Shuo Shao, Yiming Li, Yu He, Hongwei Yao, Wenyan Yang, Dacheng Tao, and Zhan Qin. Sok: Large language model copyright auditing via fingerprinting. *arXiv preprint arXiv:2508.19843*, 2025.
- Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 115–124. Association for Computational Linguistics, 2021.
- Teppe Suzuki, Ryokan Ri, and Sho Takase. Natural fingerprints of large language models. *arXiv preprint arXiv:2504.14871*, 2025.
- Qwen Team. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2, 2024.

-
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Yun-Yun Tsai, Chuan Guo, Junfeng Yang, and Laurens van der Maaten. Rofl: Robust fingerprinting of language models. *arXiv preprint arXiv:2505.12682*, 2025.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
- Jiashu Xu, Fei Wang, Mingyu Derek Ma, Pang Wei Koh, Chaowei Xiao, and Muhao Chen. Instructional fingerprinting of large language models. *arXiv preprint arXiv:2401.12255*, 2024.
- Do-hyeon Yoon, Minsoo Chun, Thomas Allen, Hans Müller, Min Wang, and Rajesh Sharma. Intrinsic fingerprint of llms: Continue training is not all you need to steal a model! *arXiv preprint arXiv:2507.03014*, 2025.
- Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. Huref: Human-readable fingerprint for large language models. *Advances in Neural Information Processing Systems*, 37:126332–126362, 2024.
- Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph Stoecklin, Heqing Huang, and Ian Molloy. Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of the 2018 on Asia conference on computer and communications security*, pp. 159–172, 2018.
- Jie Zhang, Dongrui Liu, Chen Qian, Linfeng Zhang, Yong Liu, Yu Qiao, and Jing Shao. Reef: Representation encoding fingerprints for large language models. *arXiv preprint arXiv:2410.14273*, 2024.
- Ruichong Zhang. Matrix-driven instant review: Confident detection and reconstruction of llm plagiarism on pc. *arXiv preprint arXiv:2508.06309*, 2025.
- Renjie Zhu, Xinpeng Zhang, Mengte Shi, and Zhenjun Tang. Secure neural network watermarking protocol against forging attack. *EURASIP Journal on Image and Video Processing*, 2020(1):37, 2020.

756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809

Appendix Contents

A	LLM Usage and Limitation Discussions	16
A.1	LLM Usage	16
A.2	Limitations and Future Directions	16
B	Validation of the Empirical and Analytical Null Distribution	16
B.1	Empirical True Null from Independently Initialized Models	16
B.2	Gaussian Surrogate as a Simulation-Based Approximation	16
B.3	Analytical Approximation under Weak Dependence	17
C	Theoretical Properties of Our Methods	18
C.1	Stability Analysis of Mean-Based Identity Extraction vs. Per-Sample Argmin Voting	19
C.1.1	Stability of Sample-Mean-Based Extraction	19
C.1.2	Instability of Per-Sample Argmin Voting	20
C.2	Influence of Hyperparameters	22
D	More Experiment Details	23
D.1	Implementation Details and Licenses	23
D.2	Complementary SeedPrints Results using simulation-based null	23
D.2.1	Birth-to-Lifecycle “Biometric” Fingerprinting	24
D.2.2	All-stage verifiable fingerprints	26
D.3	Cross-Size Fingerprint Within the Qwen-2.5 series	26
E	Model Catalog and Decoding Settings	26
E.1	Model list by family	27
E.1.1	Qwen-2.5-7B	27
E.1.2	Qwen2.5-14B	27
E.1.3	Llama-3.1-8B	28
E.1.4	Mistral-7B-v0.3	28
E.1.5	Gemma-2-2B	28
E.1.6	Llama-2-7B	28
E.1.7	TinyLlama-1.1B	29
E.2	Sample Results	29

A LLM USAGE AND LIMITATION DISCUSSIONS

A.1 LLM USAGE

We employed AI assistants, namely ChatGPT and Gemini, for writing and formatting support only; their use was confined to grammar and style checking, clarifying figure and table captions, and comparable surface-level edits. For programming tasks, we occasionally used GitHub Copilot and Claude as coding assistants, e.g., for code auto-completion and debugging hints. No AI assistant contributed to the scientific content, experimental design, or results reported in this paper.

A.2 LIMITATIONS AND FUTURE DIRECTIONS

Although this work uncovers seed-born, training-persistent biases that serve as reliable fingerprint signals, the proposed method operates within the white-box paradigm and thus may not apply when only restricted API access is available—for example, when responses are deterministic or safety-filtered. We situate this limitation within a broader observation: the pursuit of *true fingerprints* for LLMs remains at a very early stage even in the white-box setting, because most existing approaches rest on training-induced signatures or alignment behaviors and consequently cannot robustly protect pretrained or early-stage models. By analogy to human fingerprints, which remain stable regardless of age or environment, a *true* LLM fingerprint should persist across continued training and deployment rather than reflect incidental training artifacts—a criterion our method is expressly designed to meet. Extending this guarantee to the black-box setting, where a substantial performance gap relative to white-box methods persists, is a challenging and important open direction. We hope this work catalyzes further research toward principled, robust fingerprinting frameworks for large models.

B VALIDATION OF THE EMPIRICAL AND ANALYTICAL NULL DISTRIBUTION

Our hypothesis test in Section 4 decides whether two models share lineage by comparing the distribution of their correlation statistics over random inputs against a null distribution—the distribution of correlation statistics between two *independent models evaluated on random inputs*.

We consider two approximations to this null: (i) an empirical, simulation-based null obtained by applying the full pipeline to Gaussian surrogate outputs, and (ii) an analytical null derived from the known distribution of Kendall–Tau correlations. We justify both in three steps: characterizing the true null using independently initialized models, introducing the Gaussian surrogate as a tractable simulation-based proxy, and validating the analytical approximation against that surrogate.

B.1 EMPIRICAL TRUE NULL FROM INDEPENDENTLY INITIALIZED MODELS

The ideal null corresponds to the correlation statistics between two independent models evaluated on random inputs, and the cleanest way to instantiate this independence is to compare two independently initialized models (distinct random seeds, no training). We estimate this empirical null by evaluating ~ 2500 such pairs and computing Kendall–Tau correlations on 10,000 random inputs (as in Section 3). Figure 3 confirms that the true null is empirically a Gaussian centered at zero, exactly as our analytical treatment assumes.

B.2 GAUSSIAN SURROGATE AS A SIMULATION-BASED APPROXIMATION

Directly estimating the null from independently initialized models is computationally expensive, requiring access to many models and repeated evaluations. As a tractable alternative, we introduce a simulation-based null that uses Gaussian surrogates as proxies for model outputs under the null.

Two considerations justify this choice. First, approximating neural-network outputs with Gaussian distributions is a well-established practice in the auditing literature, supported by both empirical and theoretical evidence (Carlini et al., 2022; Lee et al., 2019). Second, our pipeline depends only on relative ordering and selection operations (rank correlations and bottom- m selection) and makes no use of semantic structure in the inputs; moreover, randomly initialized models have independent parameter matrices, so their outputs under the null exhibit no strong structured dependencies. Sampling

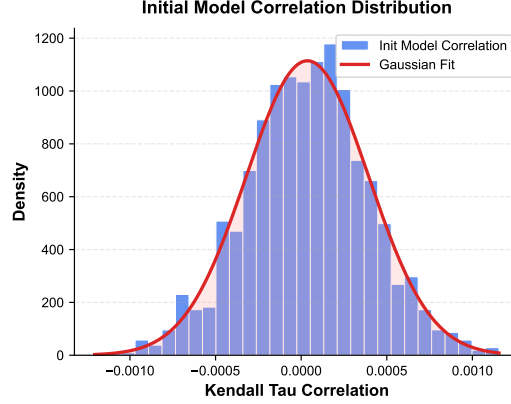


Figure 3: The empirical correlation distribution over ~ 2500 pairs of independently initialized models, evaluated on random inputs, closely matches a Gaussian distribution centered at zero, empirically validating the null model used in our test.

independent Gaussian matrices therefore constitutes a simple, faithful, and tractable proxy for the null behavior of independently initialized models.

We construct the surrogate null by sampling Gaussian matrices and applying the full pipeline, including identity-dimension selection and correlation computation:

1. Sample two Gaussian matrices of shape $\mathbb{R}^{N \times d_{\text{out}}}$,
2. Extract the most biased dimensions by selecting the bottom- m ranked average dimensions,
3. Take the intersection set $\mathcal{S}$ and compute correlations on it.

Because this construction applies the identical selection procedure to Gaussian samples, it naturally captures the dependency patterns introduced by the pipeline itself (e.g., those arising from selecting $\mathcal{S}$ across dimensions). As Figure 4 (middle) shows, the resulting distribution reproduces the centered Gaussian shape of the empirical null obtained from independently initialized models, confirming that the Gaussian surrogate is an effective simulation-based approximation of the true null.

B.3 ANALYTICAL APPROXIMATION UNDER WEAK DEPENDENCE

Although the Gaussian surrogate faithfully captures the null under the full pipeline, it still entails repeated sampling and injects randomness into the estimation. We therefore derive a maximally efficient analytical approximation that dispenses with sampling altogether.

Under the null hypothesis that the two models are independent², each Kendall–Tau correlation τ_j is approximately zero-mean with known variance σ^2 . If correlations across identity dimensions were independent, the central limit theorem would imply

$$\bar{\tau} = \frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} \tau_j \sim \mathcal{N}\left(0, \frac{\sigma^2}{|\mathcal{S}|}\right).$$

In practice, softmax normalization does introduce dependencies across dimensions. Nonetheless, Figure 4 shows that the analytical Gaussian closely matches the distribution obtained from the Gaussian surrogate pipeline across a range of settings, including varying numbers of random inputs n and output dimensions D , demonstrating that these dependencies are sufficiently weak to leave the approximation’s accuracy unaffected. The likely mechanism is that the relatively high temperature ($T = 10$) smooths the output distribution and thereby suppresses inter-dimension coupling.

Together, these results justify the analytical Gaussian null adopted in Section 4 as an efficient, deterministic, and accurate approximation that eliminates the need for costly empirical baseline construction.

²As discussed in Appendix B.2, although outputs may exhibit weak dependencies due to shared inputs, this approximation remains accurate empirically.

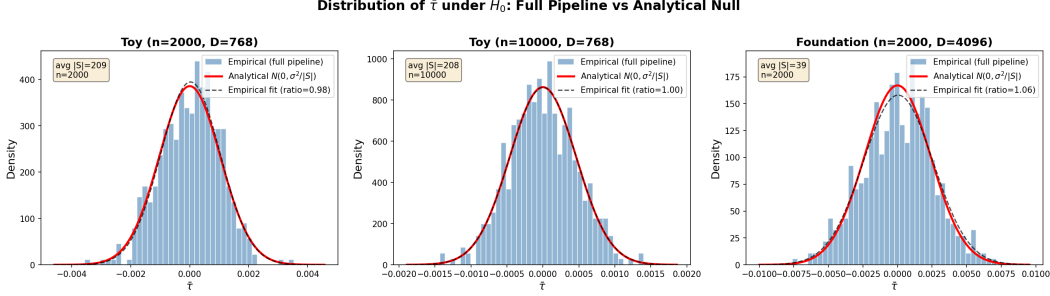


Figure 4: Validation of the analytical Gaussian null. Applying the full pipeline (including identity-dimension selection and correlation aggregation) to Gaussian surrogate outputs, we compare the resulting distribution of $\bar{\tau}$ against the analytical Gaussian $\mathcal{N}(0, \sigma^2/|S|)$. The close agreement confirms that the analytical approximation accurately captures the null behavior despite the weak dependencies introduced by the pipeline.

On variance scaling. The true null obtained from randomly initialized models has slightly smaller variance than the Gaussian null approximation, but this mismatch has no bearing on our hypothesis-test results. Our primary evaluation for the simulation-based null in Appendix D.2 uses the Mann–Whitney U -test, which is rank-based and hence invariant to any positive scaling: for any $c > 0$, $x_i < x_j \Leftrightarrow cx_i < cx_j$, so every rank statistic and rejection threshold is unchanged. For tests that do depend on absolute variance, we additionally evaluated variance-sensitive statistics such as the t -test, whose statistic scales as $t' = t/c$ when the variance decreases. Because $c < 1$ in our case, the scaling shrinks the magnitude of t and renders the test *more conservative* rather than inflating significance. Empirically, across the extensive experiments in Appendix D.2, this variance scaling leaves the power of the t -test intact, which indicates that the distributional separation between the alternative and the null is far larger than the scaled variance.

For the analytical null used in the main paper, the variance enters explicitly through the normalization of the z -score, so any scale discrepancy is absorbed in the denominator. Since the analytical variance is slightly larger than the empirical one, the resulting z -scores are correspondingly smaller, again biasing the test toward conservatism and guarding against false positives.

C THEORETICAL PROPERTIES OF OUR METHODS

Our goal is to identify the output dimensions that the initialized model is least likely to predict under random inputs. These dimensions correspond to directions the model inherently disfavors owing to initialization-induced bias—a bias independent of the input distribution—and they constitute the basis of a stable fingerprint signal. This section formalizes the two empirical extractors, proves that the mean-based extractor we adopt is stable while per-sample voting is not, and characterizes the sample complexity governing recovery.

Setup. Consider a model g with fixed parameters and random inputs x_i (uniform random tokens or random embeddings as in the main paper). For each output dimension $j \in [d_{\text{out}}]$, $g_j(x)$ is thus a random variable and its population mean is $\bar{g}_j := \mathbb{E}[g_j(x)]$. The population-level directions we aim to capture correspond to the m output dimensions with the smallest expected values, formally defined as

$$\mathcal{M} = \arg \min_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{j \in J} \bar{g}_j.$$

Let $(\bar{g}_{(1)}, \dots, \bar{g}_{(d_{\text{out}})})$ denote the sorted means in non-decreasing order, and write

$$\Gamma(m) := \bar{g}_{(m+1)} - \bar{g}_{(m)} > 0$$

for the population “gap” between the m -th and $(m+1)$ -th smallest means. This gap measures how well-separated the target set $\mathcal{M}$ is from the remaining coordinates. Importantly, for a fixed m and input distribution, $\Gamma(m)$ depends solely on the model’s inherent initialization bias. *However, increasing the length of random input sequences amplifies this bias via concentration, making the separation more pronounced at the population level.*

There are two natural empirical strategies to extract such population-level least-likely dimensions:

1. **Sample-mean-based extraction (ours).** Estimate the expectation of each output coordinate by the empirical mean over random inputs $X = \{x_i\}_{i=1}^n$ and select the m coordinates with the smallest value. Following the notation in Section 4, where g denotes the model (either the base model f or the suspect model f'):

$$\hat{g} = \frac{1}{n} \sum_{i=1}^n g(x_i), \quad \hat{\mathcal{M}}^{\text{mean}} = \arg \min_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{j \in J} \hat{g}_j.$$

2. **Per-sample argmin voting.** For each random input $x_i \in \mathbb{R}^{\ell \times d}$, select the smallest-value coordinate and then take the m most frequent dimensions:

$$j^*(x_i) = \arg \min_{j \in \{1, \dots, d_{\text{out}}\}} g_j(x_i), \quad \hat{\mathcal{M}}^{\text{vote}} = \arg \max_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{i=1}^n \mathbf{1}\{j^*(x_i) \in J\}.$$

Let $p_j := \Pr(j^*(x) = j)$ denote the population probability that index j attains the minimum, and let

$$\mathcal{M}_{\text{vote}} := \arg \max_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{j \in J} p_j$$

be the population top- m set under voting.

Both approaches are intuitively reasonable, yet they behave very differently under perturbation. As we prove below, the sample-mean-based strategy is provably stable, whereas per-sample argmin voting is discontinuous and can flip under arbitrarily small perturbations, rendering it inherently noise-sensitive. This contrast is the formal justification for our design choice to extract identity dimensions from the mean response vector.

C.1 STABILITY ANALYSIS OF MEAN-BASED IDENTITY EXTRACTION VS. PER-SAMPLE ARGMIN VOTING

C.1.1 STABILITY OF SAMPLE-MEAN-BASED EXTRACTION

We first establish that the sample-mean-based operator—which selects the m smallest coordinates of the empirical mean vector—is stable under small perturbations to model outputs. Such perturbations model the realistic variation in outputs from the *same* model, e.g., numerical noise or evaluation under different precision modes.

Formally, we write the perturbed outputs as $g'(x) = g(x) + \eta(x)$, where $\eta(x) \in \mathbb{R}^{d_{\text{out}}}$ denotes the stochastic deviation. We assume that for each sample x_i and coordinate j , the random variables $\eta_j(x_i)$ are independent across samples (not necessarily across coordinates) and σ -sub-Gaussian with $\mathbb{E}[\eta_j(x_i)] = 0$.³

Lemma C.1 (Stability of sample-mean-based extraction). *Let $\hat{g}$ and $\hat{g}'$ be the empirical means over $X = \{x_i\}_{i=1}^n$, i.e., $\hat{g} = \frac{1}{n} \sum_{i=1}^n g(x_i)$ and $\hat{g}' = \frac{1}{n} \sum_{i=1}^n g'(x_i) = \hat{g} + \frac{1}{n} \sum_{i=1}^n \eta(x_i)$. Let $\hat{\Gamma}(m)$ denote the empirical gap at rank m . Then,*

$$\Pr(\hat{\mathcal{M}}^{\text{mean}}(g') \neq \hat{\mathcal{M}}^{\text{mean}}(g)) \leq 2d_{\text{out}} \exp\left(-\frac{n\hat{\Gamma}(m)^2}{8\sigma^2}\right).$$

Interpretation. The lemma provides a quantitative upper bound on the probability that the sample-mean-based extractor changes its selected set under perturbations. This instability probability decreases exponentially with the number of averaged samples n , due to variance reduction from σ^2 per sample to σ^2/n in the empirical mean.

³This sub-Gaussian assumption is mild: bounded random variables are σ -sub-Gaussian (Hoeffding's lemma), and common initialization schemes together with bounded activations yield light-tailed behavior in both initialized and trained networks. The Gaussian case $\eta_j(x_i) \sim \mathcal{N}(0, \sigma^2)$ is a special case of this assumption.

Proof. We condition on the sample $X = \{x_i\}_{i=1}^n$ throughout the proof, so that $\hat{g}$ and $\hat{\Gamma}(m)$ are fixed, and all probabilities are taken over the randomness of $\{\eta(x_i)\}_{i=1}^n$ only.

For any coordinate j , by definition of $\hat{g}'$, $\hat{g}'_j - \hat{g}_j = \frac{1}{n} \sum_{i=1}^n \eta_j(x_i)$. Each $\eta_j(x_i)$ is mean-zero and σ -sub-Gaussian, so their average is mean-zero and $(\sigma/\sqrt{n})$ -sub-Gaussian: the variance proxy scales as $1/n$ under averaging. Hence, for any $\varepsilon > 0$, by the standard sub-Gaussian tail bound,

$$\Pr(|\hat{g}'_j - \hat{g}_j| \geq \varepsilon \mid X) \leq 2 \exp\left(-\frac{n\varepsilon^2}{2\sigma^2}\right). \quad (1)$$

Now consider the event that *any* coordinate fluctuates by more than ε :

$$\left\{ \max_{1 \leq j \leq d_{\text{out}}} |\hat{g}'_j - \hat{g}_j| \geq \varepsilon \right\}.$$

Taking a union bound over all d_{out} coordinates yields

$$\Pr\left(\max_j |\hat{g}'_j - \hat{g}_j| \geq \varepsilon \mid X\right) \leq \sum_{j=1}^{d_{\text{out}}} \Pr(|\hat{g}'_j - \hat{g}_j| \geq \varepsilon \mid X) \leq 2d_{\text{out}} \exp\left(-\frac{n\varepsilon^2}{2\sigma^2}\right). \quad (2)$$

We now show that

$$\left\{ \widehat{\mathcal{M}}^{\text{mean}}(g') \neq \widehat{\mathcal{M}}^{\text{mean}}(g) \right\} \subseteq \left\{ \max_j |\hat{g}'_j - \hat{g}_j| \geq \hat{\Gamma}(m)/2 \right\}.$$

Equivalently, if $\max_j |\hat{g}'_j - \hat{g}_j| < \hat{\Gamma}(m)/2$, then the bottom- m identity set remains unchanged.

Let $j_{(1)}, \dots, j_{(d_{\text{out}})}$ be a permutation of $\{1, \dots, d_{\text{out}}\}$ such that $\hat{g}_{j_{(1)}} \leq \hat{g}_{j_{(2)}} \leq \dots \leq \hat{g}_{j_{(d_{\text{out}})}}$. Recall that the empirical gap at rank m is $\hat{\Gamma}(m) := \hat{g}_{j_{(m+1)}} - \hat{g}_{j_{(m)}} > 0$. Denote $\widehat{\mathcal{M}}^{\text{mean}}(g) := \{j_{(1)}, \dots, j_{(m)}\}$ and $\widehat{\mathcal{M}}_c^{\text{mean}}(g) := \{j_{(m+1)}, \dots, j_{(d_{\text{out}})}\}$. For any $i \in \widehat{\mathcal{M}}^{\text{mean}}(g)$ and any $\ell \in \widehat{\mathcal{M}}_c^{\text{mean}}(g)$, $\hat{g}_\ell - \hat{g}_i \geq \hat{g}_{j_{(m+1)}} - \hat{g}_{j_{(m)}} = \hat{\Gamma}(m)$. With $|\hat{g}'_i - \hat{g}_i| < \hat{\Gamma}(m)/2$ and $|\hat{g}'_\ell - \hat{g}_\ell| < \hat{\Gamma}(m)/2$, we have

$$\hat{g}'_\ell - \hat{g}'_i \geq (\hat{g}_\ell - \frac{\hat{\Gamma}(m)}{2}) - (\hat{g}_i + \frac{\hat{\Gamma}(m)}{2}) \geq \hat{\Gamma}(m) - \hat{\Gamma}(m) = 0.$$

Thus no coordinate in $\widehat{\mathcal{M}}_c^{\text{mean}}(g)$ can become smaller than any coordinate in $\widehat{\mathcal{M}}^{\text{mean}}(g)$ under this deviation bound, and therefore

$$\widehat{\mathcal{M}}^{\text{mean}}(g') = \widehat{\mathcal{M}}^{\text{mean}}(g).$$

Since $\hat{\Gamma}(m)$ is fixed under conditioning on X , plugging $\varepsilon = \hat{\Gamma}(m)/2$ into (2) yields

$$\Pr\left(\widehat{\mathcal{M}}^{\text{mean}}(g') \neq \widehat{\mathcal{M}}^{\text{mean}}(g)\right) \leq \Pr\left(\max_j |\hat{g}'_j - \hat{g}_j| \geq \hat{\Gamma}(m)/2 \mid X\right) \leq 2d_{\text{out}} \exp\left(-\frac{n\hat{\Gamma}(m)^2}{8\sigma^2}\right). \quad \square$$

C.1.2 INSTABILITY OF PER-SAMPLE ARGMIN VOTING

In contrast, per-sample voting applies a discontinuous winner-take-all operator to each input, so arbitrarily small output perturbations can flip its decisions with non-vanishing probability. We retain the same stochastic perturbation model $g'(x) = g(x) + \eta(x)$.

Lemma C.2 (Instability of per-sample argmin voting). *For a fixed input x , let*

$$j^*(x; g) := \arg \min_j g_j(x), \quad \Delta_g(x) := \min_{j \neq j^*(x; g)} \{g_j(x) - g_{j^*(x; g)}(x)\} > 0$$

denote the argmin and its margin. Then, for any fixed x ,

$$\Pr(j^*(x; g') \neq j^*(x; g) \mid x) \geq \Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right),$$

and for n i.i.d. inputs $x_1, \dots, x_n$,

$$\Pr\left(\exists i \leq n : j^*(x_i; g') \neq j^*(x_i; g)\right) \geq 1 - (1 - \alpha)^n,$$

where $\alpha := \mathbb{E}_x \left[\Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right) \right]$.

Interpretation. Unlike mean extraction, voting does not benefit from variance reduction: each input incurs a direct hard decision based on noisy outputs. Increasing n only increases the number of opportunities for index-flips.

Proof. We first bound the per-input change probability $\Pr(j^*(x; g') \neq j^*(x; g) \mid x)$. Fix x and write $j^* := j^*(x; g)$ for brevity. By definition of the margin, $\Delta_g(x) = \min_{j \neq j^*} \{g_j(x) - g_{j^*}(x)\}$. Let $j^{(2)}$ be an index attaining this minimum, i.e., $g_{j^{(2)}}(x) - g_{j^*}(x) = \Delta_g(x)$.

Under the perturbation $g'(x) = g(x) + \eta(x)$, we have

$$g'_{j^{(2)}}(x) - g'_{j^*}(x) = [g_{j^{(2)}}(x) - g_{j^*}(x)] + [\eta_{j^{(2)}}(x) - \eta_{j^*}(x)] = \Delta_g(x) + [\eta_{j^{(2)}}(x) - \eta_{j^*}(x)].$$

Since $\eta_{j^{(2)}}(x)$ and $\eta_{j^*}(x)$ are independent $\mathcal{N}(0, \sigma^2)$ variables, their difference is Gaussian with

$$\eta_{j^{(2)}}(x) - \eta_{j^*}(x) \sim \mathcal{N}(0, 2\sigma^2).$$

The event that the argmin changes, $j^*(x; g') \neq j^*(x; g)$, certainly occurs whenever $g'_{j^{(2)}}(x) \leq g'_{j^*}(x)$. Hence,

$$\Pr(j^*(x; g') \neq j^*(x; g) \mid x) \geq \Pr(g'_{j^{(2)}}(x) \leq g'_{j^*}(x) \mid x).$$

Using the expression above,

$$\Pr(g'_{j^{(2)}}(x) \leq g'_{j^*}(x) \mid x) = \Pr(g'_{j^{(2)}}(x) - g'_{j^*}(x) \leq 0 \mid x) = \Pr(\Delta_g(x) + [\eta_{j^{(2)}}(x) - \eta_{j^*}(x)] \leq 0 \mid x).$$

Let $Z \sim \mathcal{N}(0, 1)$. Since $\eta_{j^{(2)}}(x) - \eta_{j^*}(x) \stackrel{d}{=} \sqrt{2}\sigma Z$, we obtain

$$\Pr(\Delta_g(x) + [\eta_{j^{(2)}}(x) - \eta_{j^*}(x)] \leq 0 \mid x) = \Pr\left(Z \leq -\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right) = \Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right).$$

Therefore, $\Pr(j^*(x; g') \neq j^*(x; g) \mid x) \geq \Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right)$.

Now consider n i.i.d. inputs $x_1, \dots, x_n$. Conditional on the inputs, the events

$$E_i := \{j^*(x_i; g') \neq j^*(x_i; g)\} \quad E_i^c := \{j^*(x_i; g') = j^*(x_i; g)\}$$

are independent across i , because the perturbations $\eta(x_i)$ are independent across i . Moreover,

$$\Pr(E_i \mid x_i) = \Pr(j^*(x_i; g') \neq j^*(x_i; g) \mid x) \geq \Phi\left(-\frac{\Delta_g(x_i)}{\sqrt{2}\sigma}\right) \geq \Phi\left(-\frac{\Delta_g(x_i)}{\sqrt{2}\sigma}\right).$$

Define $q(x_i) := \Pr(j^*(x_i; g') \neq j^*(x_i; g) \mid x)$, and $\alpha := \mathbb{E}_x\left[\Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right)\right]$. Taking expectations over the randomness of x_i , we obtain $\mathbb{E}[\Pr(E_i \mid x_i)] = \alpha$. Therefore, by independence across i ,

$$\Pr(E_1^c \cap \dots \cap E_n^c) = \mathbb{E}\left[\prod_{i=1}^n (1 - q(x_i))\right] \leq \mathbb{E}\left[\prod_{i=1}^n (1 - \Phi(-\Delta_g(x_i)/(\sqrt{2}\sigma)))\right] \leq (1 - \alpha)^n,$$

where the last inequality uses i.i.d. inputs and Jensen's inequality applied to the convex function $u \mapsto \log(1 - u)$ on $u \in [0, 1)$. Therefore,

$$\Pr(\exists i \leq n : j^*(x_i; g') \neq j^*(x_i; g)) = 1 - \Pr(E_1^c \cap \dots \cap E_n^c) \geq 1 - (1 - \alpha)^n.$$

Finally, note that the empirical voting set $\widehat{\mathcal{M}}^{\text{vote}}$ is determined by the empirical counts of the dimensions $\{j^*(x_i; g)\}_{i=1}^n$. If the empirical gap between the m -th and $(m+1)$ -th most frequent dimensions is of order $O(1/n)$ (i.e., only a constant number of votes), then a change in $j^*(x_i)$ for a single input x_i is sufficient to swap the ordering across this boundary and thus alter the top- m set. Combining this observation with the lower bound $1 - (1 - \alpha)^n$ shows that, in such regimes, the probability that $\widehat{\mathcal{M}}^{\text{vote}}(g') \neq \widehat{\mathcal{M}}^{\text{vote}}(g)$ is bounded away from zero uniformly in n , i.e., per-sample argmin voting does not enjoy variance reduction with more samples. $\square$

C.2 INFLUENCE OF HYPERPARAMETERS

We now quantify how many random inputs the empirical mean estimator requires to recover the true bias. Reusing the notation from **Setup**, each output coordinate $g_j(x)$ is a random variable under random inputs $x \sim \mathcal{D}$, with population mean $\bar{g}_j := \mathbb{E}[g_j(x)]$. $(\bar{g}_{(1)}, \dots, \bar{g}_{(d_{\text{out}})})$ represents the sorted means in non-decreasing order, and $\gamma := \bar{g}_{(m+1)} - \bar{g}_{(m)} > 0$ is the separation margin. We estimate $\bar{g}$ using n i.i.d. random inputs:

$$\hat{g}_j := \frac{1}{n} \sum_{i=1}^n g_j(x_i), \quad \hat{g} = (\hat{g}_1, \dots, \hat{g}_{d_{\text{out}}}).$$

The target dimensions set is $\mathcal{M}$ with size m and our empirical estimation gives $\widehat{\mathcal{M}}$.

Lemma C.3 (Sufficient condition for recovering the true set). *If $\|\hat{g} - \bar{g}\|_\infty \leq \varepsilon$ and $2\varepsilon < \gamma$, then the empirical bottom- m set equals the true set: $\widehat{\mathcal{M}} = \mathcal{M}$.*

Proof. For any $j \in \mathcal{M}$ and $j' \notin \mathcal{M}$, we have $\bar{g}_j \leq \bar{g}_{(m)}$ and $\bar{g}_{j'} \geq \bar{g}_{(m+1)}$. The ℓ_∞ bound gives

$$\hat{g}_j \leq \bar{g}_{(m)} + \varepsilon, \quad \hat{g}_{j'} \geq \bar{g}_{(m+1)} - \varepsilon.$$

Thus,

$$\hat{g}_j - \hat{g}_{j'} \leq -(\gamma - 2\varepsilon) < 0,$$

so no outside index can enter nor inside index leave the bottom- m set. $\square$

We next translate this recovery condition into hyperparameter terms, adopting the same sub-Gaussian assumption as in Appendix C.1. The assumption is mild: by Hoeffding's inequality, any bounded random variable is σ -sub-Gaussian with σ proportional to its range, and in our setting each $g_j(x)$ is a logit or final hidden activation, quantities that are typically bounded or light-tailed in both initialized networks (owing to controlled parameter initialization) and trained networks.

Corollary C.4 (Sample complexity under sub-Gaussian coordinates). *Suppose each $g_j(x)$ is σ -sub-Gaussian. Then with probability at least $1 - \delta$,*

$$\|\hat{g} - \bar{g}\|_\infty \leq \sigma \sqrt{\frac{2 \log(2d_{\text{out}}/\delta)}{n}}.$$

Consequently, if

$$n \geq \frac{8\sigma^2}{\gamma^2} \log\left(\frac{2d_{\text{out}}}{\delta}\right),$$

then $\widehat{\mathcal{M}} = \mathcal{M}$ with probability at least $1 - \delta$.

Proof. For fixed j , sub-Gaussian concentration implies

$$\Pr(|\hat{g}_j - \bar{g}_j| > \varepsilon) \leq 2 \exp(-n\varepsilon^2/2\sigma^2).$$

A union bound over all d_{out} coordinates yields

$$\Pr\left(\|\hat{g} - \bar{g}\|_\infty > \varepsilon\right) \leq \sum_{j=1}^{d_{\text{out}}} \Pr(|\hat{g}_j - \bar{g}_j| > \varepsilon) \leq 2d_{\text{out}} \exp\left(-\frac{n\varepsilon^2}{2\sigma^2}\right).$$

Set the right-hand side to δ and solve for ε to obtain, with probability at least $1 - \delta$,

$$\|\hat{g} - \bar{g}\|_\infty \leq \sigma \sqrt{\frac{2 \log(2d_{\text{out}}/\delta)}{n}}.$$

Now choose

$$\varepsilon := \sigma \sqrt{\frac{2 \log(2d_{\text{out}}/\delta)}{n}}.$$

If $n \geq \frac{8\sigma^2}{\gamma^2} \log\left(\frac{2d_{\text{out}}}{\delta}\right)$ (i.e., if $2\varepsilon < \gamma$), then Lemma C.3 applies and implies $\widehat{\mathcal{M}} = \mathcal{M}$ with probability at least $1 - \delta$. $\square$

Table 9: Fingerprint behavior under different random initializations across model families: every distinct-seed pair yields $p > 0.01$ under both tests and both output types, so SeedPrints separates seeds reliably.

(a) LLaMA-style models					(b) Qwen-style models				
Seed Pair	Logits Output		Hidden State		Seed Pair	Logits Output		Hidden State	
	<i>t</i> -test	<i>U</i> -test	<i>t</i> -test	<i>U</i> -test		<i>t</i> -test	<i>U</i> -test	<i>t</i> -test	<i>U</i> -test
s_{42} vs. s_{2000}	0.404	0.456	0.357	0.532	s_{123} vs. s_{1000}	0.741	0.727	0.094	0.074
s_{123} vs. s_{42}	0.214	0.295	0.678	0.565	s_{1000} vs. s_{123}	0.954	0.971	0.125	0.094
s_{1000} vs. s_{123}	0.219	0.246	0.363	0.335	s_{42} vs. s_{2000}	0.273	0.360	0.451	0.529
s_{2000} vs. s_{1000}	0.282	0.291	0.434	0.481	s_{2000} vs. s_{42}	0.215	0.206	0.230	0.295

Table 10: (LLaMA-style models) Trained models retain the fingerprint of their initialization (p -value < 0.01 across both output types and both tests), whereas none of the four baselines detects the shared lineage.

Model Pair	Logits Output		Hidden State		Baselines			
	<i>t</i> -test	<i>u</i> -test	<i>t</i> -test	<i>u</i> -test	Intrinsic	REEF	PCS	ICS
s_{42}^{init} vs. s_{42}^{base}	3.33e-3✓	1.02e-3✓	2.20e-8✓	6.28e-8✓	-0.021×	0.375×	0.580×	0.196×
s_{123}^{init} vs. s_{123}^{base}	2.06e-3✓	7.33e-3✓	7.09e-6✓	1.37e-5✓	0.149×	0.369×	0.581×	0.188×
s_{1000}^{init} vs. s_{1000}^{base}	2.44e-3✓	4.14e-3✓	5.58e-4✓	2.81e-3✓	-0.252×	0.381×	0.581×	0.188×
s_{2000}^{init} vs. s_{2000}^{base}	5.63e-3✓	6.76e-3✓	4.00e-10✓	1.27e-9✓	-0.337×	0.331×	0.581×	0.188×

Consequently, accurate recovery requires $n = \Omega\left(\frac{\log d_{\text{out}}}{\gamma^2}\right)$: the number of random queries grows only logarithmically with the output dimension but scales inversely with the squared separation margin γ . The margin γ is in turn shaped by two factors:

- **Intrinsic bias strength.** Longer random input sequences amplify initialization-induced bias through concentration, widening the population separation between coordinates.
- **Choice of m .** Different values of m probe different regions of the ranked spectrum $(\bar{g}_{(1)}, \dots, \bar{g}_{(d_{\text{out}})})$ and thus yield different effective margins in practice.

The former is controlled through the input length l , whereas the latter is selected empirically from the observed gaps in the mean spectrum.

D MORE EXPERIMENT DETAILS

D.1 IMPLEMENTATION DETAILS AND LICENSES

We train all models with the Hugging Face Transformers Trainer (Wolf et al., 2020) and use Accelerate (Gugger et al., 2022) for distributed runs. All open-source models are loaded from their official Hugging Face releases and used under their original licenses—Llama models under the Meta Llama Community License and the remainder under Apache-2.0—and all datasets are obtained via the Hugging Face Datasets library (itself Apache-2.0), with dataset content governed by each dataset’s stated license.

D.2 COMPLEMENTARY SEEDPRINTS RESULTS USING SIMULATION-BASED NULL

This section provides the complementary results for Section 5. We report results using the empirical simulation-based null of Algorithm 1 with both *logits* and *hidden state* outputs, and we evaluate under both the one-sided *t*-test (*t*-test) and the Mann–Whitney *U* test (*U* test); across every configuration the conclusions of the main paper are reproduced.

Table 11: (Qwen-style models) Trained models retain the fingerprint of their initialization (p -value < 0.01 across both output types and both tests), corroborating the LLaMA-style results.

Model Pair	Logits Output		Hidden State	
	t -test	U -test	t -test	U -test
s_{123}^{init} vs. s_{123}^{base}	2.38e-03	1.68e-03	7.36e-15	3.38e-13
s_{1000}^{init} vs. s_{1000}^{base}	1.35e-04	4.84e-05	4.41e-13	2.01e-11
s_{42}^{init} vs. s_{42}^{base}	1.39e-03	1.46e-03	1.06e-24	2.05e-19
s_{2000}^{init} vs. s_{2000}^{base}	1.80e-03	1.28e-03	4.87e-24	1.92e-20

Table 12: Fingerprint persistence under continual training on diverse datasets (base model: seed 1000, corpus openwebtext); the U -test denotes the Mann–Whitney U test. SeedPrints attributes lineage correctly on both TinyStories and the sharply shifted code corpus (The Stack) under logits and hidden states alike, while the baselines fail on The Stack.

(a) LLaMA-style models

Setting	Ours (logits)		Ours (hidden)		Baselines			
Continual corpus (seed)	t -test	U -test	t -test	U -test	Intrinsic	REEF	PCS	ICS
TinyStories (1000)	0 [✓]	0 [✓]	0 [✓]	7.77e-89 [✓]	1.000 [✓]	0.759 [×]	0.999 [✓]	0.996 [✓]
TinyStories (123)	1.000 [✓]	1.00 [✓]	0.943 [✓]	0.902 [✓]	0.950 [×]	0.658 [✓]	0.332 [✓]	0.012 [✓]
the_stack (1000)	0 [✓]	1.73e-287 [✓]	0 [✓]	3.09e-69 [✓]	0.489 [×]	0.557 [×]	0.585 [×]	0.123 [×]
the_stack (123)	0.616 [✓]	0.479 [✓]	0.732 [✓]	0.831 [✓]	0.445 [✓]	0.580 [✓]	0.301 [✓]	0.026 [✓]

(b) Qwen-style models

Setting	Ours (logits)		Ours (hidden)		Baselines			
Continual corpus (seed)	t -test	U -test	t -test	U -test	Intrinsic	REEF	PCS	ICS
TinyStories (1000)	5.36e-09 [✓]	1.92e-07 [✓]	8.49e-214 [✓]	5.09e-71 [✓]	1.000 [✓]	0.957 [✓]	0.999 [✓]	0.996 [✓]
TinyStories (123)	0.434 [✓]	0.433 [✓]	0.256 [✓]	0.065 [✓]	0.913 [×]	0.199 [✓]	0.328 [✓]	0.039 [✓]
the_stack (1000)	0 [✓]	4.16e-237 [✓]	1.16e-211 [✓]	2.30e-76 [✓]	0.999 [✓]	0.313 [×]	0.995 [✓]	0.976 [✓]
the_stack (123)	0.999 [✓]	0.993 [✓]	0.610 [✓]	0.491 [✓]	0.916 [×]	0.255 [✓]	0.328 [✓]	0.038 [✓]

D.2.1 BIRTH-TO-LIFECYCLE “BIOMETRIC” FINGERPRINTING

We train 12-layer, 12-head LLaMA-style models (Touvron et al., 2023) with RoPE (Su et al., 2021) and Qwen-style models (Team, 2024) from scratch. Because the simulation-based random baseline is stochastic, we report p -values averaged over 10 independent trials at a significance level of $\alpha = 0.01$. We stress that the absolute magnitude of extremely small p -values carries no additional meaning: once p drops below the floor set by numerical and sampling noise (e.g., $< 10^{-20}$), a value such as 10^{-260} is not stronger evidence than 10^{-20} —both decisively reject the null.

Different initialization seeds produce distinct fingerprints. Table 9a and Table 9b report p -values from our correlation test between pairs of models initialized with different random seeds (42, 123, 1000, and 2000) under the t -test and U -test, for LLaMA-style and Qwen-style models respectively. Every p -value is > 0.01 , so our method reliably separates models originating from different seeds: distinct seeds yield distinct fingerprint behaviors, and models are separable “at birth.”

Training preserves the initialization fingerprint. Table 10 (LLaMA-style) and Table 11 (Qwen-style) compare each initialization model s^{init} with its descendant s^{base} trained on OpenWebText (Gokaslan et al., 2019) (≈ 10 B tokens). Across all seed–model pairs the p -values are uniformly < 0.01 , showing that the bias profiles remain strongly correlated and share a common lineage—the trained model inherits the fingerprint of its initialization, and training does not erase it. The baselines in Table 10 behave oppositely: without exception they fail to distinguish across seeds, confirming that their separability originates in training-induced artifacts rather than in initialization.

Table 13: Identical data and training order do not force fingerprint behaviors to coincide across different initializations.

(a) LLaMA-style models					(b) Qwen-style models				
Model Pair	Logits Output		Hidden State		Model Pair	Logits Output		Hidden State	
	<i>t</i> -test	<i>U</i> -test	<i>t</i> -test	<i>U</i> -test		<i>t</i> -test	<i>U</i> -test	<i>t</i> -test	<i>U</i> -test
s_{123}^{init} vs. s_{1000}^{base}	0.484	0.500	0.385	0.486	s_{123}^{init} vs. s_{1000}^{base}	0.598	0.638	0.286	0.254
s_{1000}^{init} vs. s_{2000}^{base}	0.946	0.956	0.135	0.096	s_{1000}^{init} vs. s_{123}^{base}	0.804	0.805	0.236	0.240
s_{42}^{init} vs. s_{123}^{base}	0.598	0.589	0.426	0.337	s_{42}^{init} vs. s_{2000}^{base}	0.589	0.608	0.226	0.2043
s_{2000}^{init} vs. s_{42}^{base}	0.756	0.781	0.388	0.287	s_{2000}^{init} vs. s_{42}^{base}	0.523	0.482	0.312	0.323

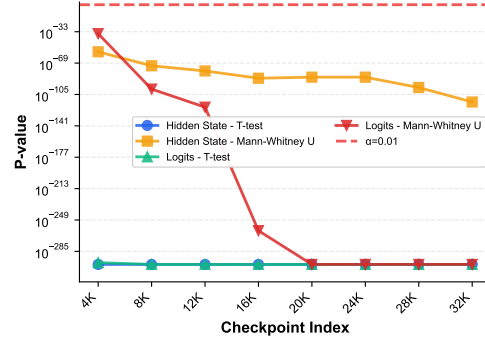
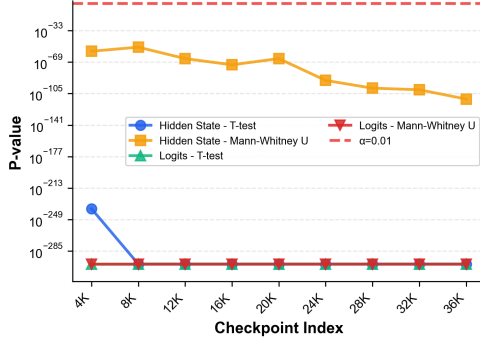


Figure 5: SeedPrints verifies lineage at every checkpoint (p -values < 0.01) and does so consistently across model families.

Identical data and order do not force fingerprints to converge. In Table 13a (LLaMA-style) and Table 13b (Qwen-style), all four “suspicious” models s_i^{base} for $i \in \{42, 123, 100, 2000\}$ are trained on *exactly the same corpus* (OpenWebText) and *in the same data order*—we fix the training seed to lock the curriculum—so that their sole difference is the initialization seed i . The aim is to test whether identical data and order could erase or confound the fingerprint. Across all cross-seed pairs the p -values remain > 0.01 , in sharp contrast to the near-zero values of Table 10: the fingerprint stays seed-specific even under identical data and curriculum.

Continual training on diverse datasets does not confound the fingerprint. The experiments above serve only to establish that SeedPrints behaves as a biometric fingerprint. From a copyright standpoint, the inability of prior methods to separate models by seed is not itself a shortcoming, since initialization seeds carry no clear copyright status; their genuine fragility lies elsewhere—their attribution is easily misled by *data distribution*, *failing to recognize lineage when the training distribution shifts substantially during continued training*.

In Table 12a and Table 12b, we continue training a base model (seed 1000, pretrained on OpenWebText (Gokaslan et al., 2019)) on two deliberately dissimilar corpora: TinyStories (Eldan & Li, 2023) (synthetic children’s stories) and The Stack (Kocetkov et al., 2022) (permissively licensed GitHub code). We contrast (i) true descendants trained from the base against (ii) *distractors* derived from a different base model (initialized with seed 123 and trained with a different data order on OpenWebText) and then continually trained on the *same* corpus, so that the decisive question is whether an attribution method can identify which descendant genuinely shares lineage with the base.

Every prior baseline fails in the code setting (The Stack), misclassifying true descendants as distractors—clear evidence that these methods track *domain* similarity rather than lineage identity: TinyStories remains close in distribution to the pretraining corpus (OpenWebText), whereas The Stack diverges sharply, and such a pronounced shift suffices to defeat detection. Our method, by contrast, attributes lineage correctly across both corpora. SeedPrints is therefore not a proxy for data distribution: it withstands substantial domain shift and persists well beyond the initial pretraining stage.

Table 14: Cross-size similarity between Qwen-2.5-14B (target) and other Qwen-2.5 models. SeedPrint reports p -values ($\uparrow$ desirable here, since none of the smaller models is a true descendant), while REEF and HUREF report similarity scores.

	Qwen-2.5-0.5B	Qwen-2.5-1.5B	Qwen-2.5-3B	Qwen-2.5-7B
SeedPrint ($\uparrow$)	0.7184	0.9707	0.3898	0.5429
REEF ($\downarrow$)	0.8250	0.8399	0.8833	0.8461
HUREF ($\downarrow$)	-6.37×10^{-5}	-2.84×10^{-5}	-8.70×10^{-6}	-6.08×10^{-6}

D.2.2 ALL-STAGE VERIFIABLE FINGERPRINTS

Controlled pretraining trajectory. We first conduct a controlled pretraining experiment on OpenWebText, in which the initialization seed is known. Treating each intermediate checkpoint as the base, we test whether our method reliably identifies its offspring along the same training trajectory. Every variant consistently recognizes the suspect model as same-lineage, with p -values remaining below the 0.01 threshold (Figure 5a for LLaMA-style models and Figure 5b for Qwen-style models), confirming the stability of SeedPrints under continuous optimization.

D.3 CROSS-SIZE FINGERPRINT WITHIN THE QWEN-2.5 SERIES

To examine whether SeedPrints “collapses” into a single family-level signature when the parameter size is varied, we report cross-size similarity between Qwen-2.5-14B (target) and smaller models drawn from the same Qwen-2.5 family. Table 14 presents the results for SeedPrint ($\uparrow$) alongside two baselines, REEF ($\downarrow$) and HUREF ($\downarrow$).

Concretely, to test whether initialization-born fingerprints collapse to a family-level signature under scaling, we conduct a cross-size study within the Qwen-2.5 series: we treat Qwen-2.5-14B as the protected target model and Qwen-2.5-{0.5B, 1.5B, 3B, 7B} as suspect models that share the architectural family but are not trained as descendants of Qwen-2.5-14B. As established in the main paper, SeedPrints returns a p -value for the hypothesis “the suspect shares initialization-born fingerprints with the target,” whereas REEF and HUREF are similarity-based baselines that emit continuous scores and require the user to fix a threshold.

Several observations stand out. First, every SeedPrints p -value lies comfortably above a typical significance level (e.g., $\alpha=0.01$), so under our hypothesis-testing view none of the smaller Qwen-2.5 models is flagged as sharing lineage with Qwen-2.5-14B. This is exactly the desired behavior: even though these models belong to the same family and reuse a similar architectural design, the test does not collapse to a single family-level signature that would spuriously conflate distinct pretraining runs.

Second, the cross-size p -values populate different regions of $[0, 1]$ rather than clustering around a single value. We do *not* read these magnitudes as a continuous “similarity score”—their role is to support a calibrated yes/no decision at a fixed significance level—but the spread does show that our test statistic remains sensitive to the specific combination of architecture and initialization seed, rather than degenerating into a generic pattern common to all Qwen-2.5 variants.

Finally, the similarity-based baselines behave markedly differently. REEF produces scores in a narrow band (0.8250–0.8833) across all sizes, and HUREF outputs values numerically close to zero (on the order of 10^{-5}). Because neither method supplies a built-in decision rule, operators must hand-tune thresholds, and it is unclear how any single cutoff could robustly separate cross-size variants from genuinely lineage-related models. SeedPrints instead yields a statistical decision at a prescribed significance level, giving a clearer and more operational answer to the question “do these two models plausibly share a training lineage?” within a model family.

E MODEL CATALOG AND DECODING SETTINGS

Our evaluation covers 58 models in total. Below we document the decoding configuration used throughout all runs and enumerate the model catalog grouped by family; for each derived model we additionally record the post-training transformation applied (*finetune*, *adapter*, *merge*, *quantization*,

or *distillation*). Throughout, “PT” and “IT” denote *pre-trained* and *instruction-tuned* source models, respectively.

Parameter	Value
max_new_tokens	512
temperature	0.7
top_p	0.9
top_k	50
do_sample	true
max_input_length	512

E.1 MODEL LIST BY FAMILY

E.1.1 QWEN-2.5-7B

Variant	HuggingFace repo	Type
PT	Qwen/Qwen2.5-7B	—
IT	Qwen/Qwen2.5-7B-Instruct	—
Derived	Qwen/Qwen2.5-Math-7B	finetune
Derived	Qwen/Qwen2.5-Coder-7B-Instruct	finetune
Derived	WangCa/Qwen2.5-7B-Medicine	finetune
Derived	huihui-ai/Qwen2.5-7B-Instruct-abliterated-v2	finetune
Derived	Locutusque/StockQwen-2.5-7B	merge
Derived	bunnycore/QevaCoT-7B-Stock	merge
Derived	fangcaotank/task-10-Qwen-Qwen2.5-7B-Instruct	adapter
Derived	SeeFlock/task-12-Qwen-Qwen2.5-7B-Instruct	adapter
Derived	Qwen/Qwen2.5-7B-Instruct-GPTQ-Int4	quantization
Derived	Qwen/Qwen2.5-7B-Instruct-GPTQ-Int8	quantization
Derived	Lansechen/Qwen2.5-7B-Open-R1-Distill	distillation

E.1.2 QWEN2.5-14B

Variant	HuggingFace repo	Type
PT	Qwen/Qwen2.5-14B	—
IT	Qwen/Qwen2.5-14B-Instruct	—
Derived	Qwen/Qwen2.5-Coder-14B	finetune
Derived	oxyapi/oxy-1-small	finetune
Derived	v000000/Qwen2.5-14B-Gutenberg-Instruct-Slerpeno	merge
Derived	ToastyPigeon/qwen-story-test-qlora	adapter
Derived	Qwen/Qwen2.5-14B-Instruct-GPTQ-Int4	quantization
Derived	deepseek-ai/DeepSeek-R1-Distill-Qwen-14B	distillation

E.1.3 LLAMA-3.1-8B

Variant	HuggingFace repo	Type
PT	meta-llama/Llama-3.1-8B	—
IT	meta-llama/Llama-3.1-8B-Instruct	—
Derived	ValiantLabs/Llama3.1-8B-Fireplace2	finetune
Derived	RedHatAI/Llama-3.1-8B-tldr	finetune
Derived	proxectonos/Llama-3.1-Carballo	finetune
Derived	mlabonne/Meta-Llama-3.1-8B-Instruct-abliterated	finetune
Derived	gaverfraxz/Meta-Llama-3.1-8B-Instruct-HalfAbliterated-TIES	merge
Derived	Xiaojian9992024/Llama3.1-8B-ExtraMix	merge
Derived	LlamaFactoryAI/Llama-3.1-8B-Instruct-cv-job-description-matching	adapter
Derived	chchen/Llama-3.1-8B-Instruct-PsyCourse-fold7	adapter
Derived	iqbalamo93/Meta-Llama-3.1-8B-Instruct-GPTQ-Q_8	quantization
Derived	DaraV/LLaMA-3.1-8B-Instruct-INT4-GPTQ	quantization
Derived	asas-ai/Llama-3.1-8B-Instruct-Open-R1-Distill	distillation

E.1.4 MISTRAL-7B-v0.3

Variant	HuggingFace repo	Type
PT	mistralai/Mistral-7B-v0.3	—
IT	mistralai/Mistral-7B-Instruct-v0.3	—
Derived	KurmaAI/AQUA-7B	finetune
Derived	openfoodfacts/spellcheck-mistral-7b	finetune
Derived	grimjim/Mistral-7B-Instruct-demi-merge-v0.3-7B	merge
Derived	chaymaemrhrioui/mistral-Brain_Model_ACC_Trainer	adapter
Derived	RedHatAI/Mistral-7B-Instruct-v0.3-GPTQ-4bit	quantization
Derived	eganwo/mistral7b-distilled-from-deepseek-r1-qwen32b	distillation

E.1.5 GEMMA-2-2B

Variant	HuggingFace repo	Type
PT	google/gemma-2-2b	—
IT	google/gemma-2-2b-it	—
Derived	rinna/gemma-2-baku-2b	finetune
Derived	anakin87/gemma-2-2b-neogenesis-ita	finetune
Derived	vonjack/gemma2-2b-merged	merge
Derived	google-cloud-partnership/gemma-2-2b-it-lora-sql	adapter
Derived	qilowoq/gemma-2-2B-it-4Bit-GPTQ	quantization
Derived	Syed-Hasan-8503/Gemma-2-2b-it-distilled	distillation

E.1.6 LLAMA-2-7B

Variant	HuggingFace repo	Type
PT	meta-llama/Llama-2-7b-hf	—
IT	meta-llama/Llama-2-7b-chat-hf	—
Derived	allenai/tulu-2-7b	finetune
Derived	QIAIUNCC/EYE-Llama_qa	finetune
Derived	DevQuasar/coma-7B-v0.1	merge
Derived	Ammar-1/llama2-Better-Tune	adapter
Derived	TheBloke/Llama-2-7B-Chat-GPTQ	quantization
Derived	cygu/llama-2-7b-logit-watermark-distill-kgw-k1-gamma0.25-delta2	distillation

E.1.7 TINYLLAMA-1.1B

Variant	HuggingFace repo	Type
PT/IT	TinyLlama/TinyLlama-1.1B-Chat-v1.0	—
Derived	alexredna/TinyLlama-1.1B-Chat-v1.0-reasoning-v2	finetune
Derived	Edentns/DataVortexTL-1.1B-v0.1	finetune
Derived	appvoid/dot-v2.7	merge
Derived	barissglc/tinyllama-tarot-v1	adapter
Derived	TheBloke/TinyLlama-1.1B-Chat-v1.0-GPTQ	quantization
Derived	anudaw/distilled-code-llama	distillation

Notes. “Type” denotes the post-training transformation applied relative to the PT/IT source model: *finetune* covers supervised and preference-optimization variants; *adapter* covers LoRA/QLoRA-style modules; *merge* covers model soups and TIES-style merges; *quantization* covers GPTQ/INTx variants; and *distillation* covers student models distilled from reasoning-augmented teachers.

E.2 SAMPLE RESULTS

We present partial results from a single LeafBench evaluation run in Table 15 (*u*-test with the simulation-based null); the reported metric is $1 - p$, used as a similarity score. We fix the significance level for *p*-values at 0.01, and, empirically, LeafBench identifies an optimal decision threshold of 0.9920, which further corroborates the reliability of our method.

Crucially, although individual *p*-values should not be read as linear similarity measures, their distribution across model pairs is highly informative: same-lineage models yield markedly lower *p*-values, whereas unrelated models remain approximately uniform—precisely the separation that a calibrated test exploits.

1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619

Model	Qwen-2.5-7B	Qwen-2.5-7B-Instruct	Qwen2.5-14B	Qwen2.5-14B-Instruct
Qwen-2.5-7B	1.000	1.000	0.079	0.000
Qwen-2.5-7B-Instruct	1.000	1.000	0.813	0.536
Qwen2.5-7B-Math	1.000	0.994	0.047	0.987
Qwen2.5-7B-Coder	0.996	0.999	0.177	0.002
Qwen2.5-7B-Instruct-Medicine	1.000	1.000	0.576	0.369
Qwen2.5-7B-Instruct-Abiliterated	1.000	1.000	0.276	0.886
Qwen2.5-7B-Stock	1.000	1.000	0.939	0.225
QevaCoT-7B	1.000	1.000	0.199	0.119
Qwen2.5-7B-Instruct-Task-10	1.000	1.000	0.672	0.090
Qwen2.5-7B-Instruct-Task-12	1.000	1.000	0.155	0.987
Qwen2.5-7B-Instruct-Int4	1.000	1.000	0.827	0.961
Qwen2.5-7B-Instruct-Int8	1.000	1.000	0.914	0.739
Qwen2.5-7B-Open-R1-Distill	1.000	1.000	0.302	0.955
Qwen2.5-14B	0.324	0.487	1.000	1.000
Qwen2.5-14B-Instruct	0.403	0.702	1.000	1.000
Qwen2.5-Coder-14B	0.492	0.660	1.000	1.000
oxy-1-small	0.492	0.104	1.000	1.000
Qwen2.5-14B-Gutenberg-Instruct-Slerpeno	0.856	0.931	1.000	1.000
Qwen-story-test-qlora	0.112	0.898	1.000	1.000
Qwen2.5-14B-Instruct-GPTQ-Int4	0.583	0.092	1.000	1.000
DeepSeek-R1-Distill-Qwen-14B	0.204	0.000	1.000	1.000
Llama-3.1-8B	0.638	0.714	0.156	0.000
Llama-3.1-8B-Instruct	0.643	0.118	0.128	0.510
Llama-3.1-8B-Fireplace2	0.715	0.532	0.065	0.354
Llama-3.1-8B-TLDR	0.925	0.811	0.331	0.014
Llama-3.1-8B-Carballo	0.001	0.605	0.003	0.946
Llama-3.1-8B-Instruct-Abiliterated	0.061	0.005	0.121	0.422
Llama-3.1-8B-Instruct-HalfAbiliterated-TIES	0.356	0.509	0.796	0.000
Llama-3.1-8B-ExtraMix	0.640	0.160	0.721	0.875
Llama-3.1-8B-Instruct-cv-job-description-matching	0.684	0.292	0.004	0.001
Llama-3.1-8B-Instruct-PsyCourse-fold7	0.682	0.057	0.063	0.191
Llama-3.1-8B-Instruct-8bit	0.798	0.214	0.154	0.385
Llama-3.1-8B-Instruct-4bit	0.513	0.023	0.005	0.933
Llama-3.1-8B-Instruct-Open-R1-Distill	0.800	0.101	0.772	0.024
Mistral-7B-v0.3	0.224	0.352	0.506	0.044
Mistral-7B-v0.3-Instruct	0.000	0.000	0.984	0.104
AQUA-7B	0.014	0.613	0.703	0.569
Mistral-7B-v0.3-Spellcheck	0.005	0.481	0.872	0.105
Mistral-7B-v0.3-Instruct-demi-merge	0.904	0.774	0.058	0.522
Mistral-7B-v0.3-Brain	0.340	0.983	0.481	0.016
Mistral-7B-v0.3-Instruct-GPTQ-4bit	0.002	0.281	0.089	0.784
Mistral-7B-distilled-from-deepseek-r1-qwen32b	0.009	0.001	0.511	0.015
Gemma-2-2b	0.747	0.869	0.689	0.000
Gemma-2-2b-it	0.944	0.039	0.626	0.060
Gemma-2-baku-2b	0.616	0.231	0.176	0.464
Gemma-2-2b-neogenesis-ita	0.002	0.536	0.079	0.980
Gemma-2-2b-merged	0.951	0.012	0.651	0.075
Gemma-2-2b-it-lora-sql	0.236	0.408	0.000	0.000
Gemma-2-2B-it-4Bit-GPTQ	0.945	0.159	0.000	0.008
Gemma-2-2b-it-distilled	0.482	0.573	0.271	0.276
Llama-2-7b	0.443	0.470	0.148	0.276
Llama-2-7b-chat	0.528	0.868	0.424	0.766
tulu-2-7b	0.125	0.272	0.170	0.888
EYE-Llama_qa	0.657	0.244	0.674	0.968
coma-7B-v0.1	0.953	0.004	0.573	0.731
llama2-Better-Tune	0.136	0.699	0.469	0.860
Llama-2-7B-Chat-GPTQ	0.594	0.603	0.582	0.424
llama-2-7b-logit-watermark-distill-kgw-k1-gamma0.25-delta2	0.349	0.224	0.366	0.976

Table 15: **Sample results.** Pairwise similarity ($1 - p$) between target models (rows) and base models (columns). Cells with $1 - p \geq 0.99$ are highlighted in green as same-lineage pairs; the green block structure shows that high scores concentrate precisely on the true within-family lineage pairs.